CRYPTO-CURRENCY LIQUIDITY POOLS WITH OFF-CHAIN DATA INTEGRATION FOR MITIGATING IMPERMANENT LOSS

Michel, Liam (UG - Comp Sci & Elec Eng)
Lm01443@surrey.ac.uk

Liam Michel 6641319

Declaration of Originality

"'I confirm that the submitted work is my own work and that I have clearly identified and fully acknowledged all material that is entitled to be attributed to others (whether published or unpublished) using the referencing system set out in the programme handbook. I agree that the University may submit my work to means of checking this, such as the plagiarism detection service Turnitin® UK. I confirm that I understand that assessed work that has been shown to have been plagiarised will be penalised."

Contents

Declaration of Originality 1
Abstract 4
1. Introduction 5
1.1 Introduction 5
1.2 Background 5
1.3 Background Key Knowledge 6
1.4 Project Description 9
1.5 Project Aims and Objectives 9
1.6 Possible Issues 10
1.7 Summary 10
2. Literature Review 10
2.1 Methods to Buy Cryptocurrencies. 10
2.1.1 Centralized Exchanges 10
2.1.2 DeX's (Decentralized Exchanges) 11
2.2 DeX Implementations (Liquidity Pools) 11
2.2.1 Order book model – Why it doesn't work with blockchain. 11
2.2.2 The Liquidity Pool approach 11
2.2.2.1 Liquidity Provision 12
2.2.2.2 Burning LP Tokens 12
2.3 Automated Market Makers (AMMs) and existing algorithms for implementations 12
2.3.1 Constant Sum Market Maker (CSMM) 12
2.3.2 Constant Product Automated Market Maker (CPMM) 13
2.3.3 CPMM and the need and utility of arbitrageurs. 16
2.4 Impermanent Loss 17
2.4.1 Impermanent Loss rough example / explanation 17
2.4.2 Appropriate token pairs for liquidity pools 17
What are Stablecoins? 17
2.4 Smart Contracts (Solidity) and Security for Ethereum Blockchain 18
2.4.1 Smart contract ownership / permissions. 18
2.5 Addressing Impermanent Loss 19
2.6 Oracles for off-chain data aggregation 19
2.6.1 What are Oracles? 19
2.6.2 Off-chain data integration example 20
2.6.3 Why centralized oracles do not work. 20
2.7 Chainlink for secure off-chain data aggregation 20

2.8 Use of Chainlink for my implementation and how to interact with Chainlink. 21
2.9 Liquidity Locking in liquidity pools. 21
3. Problem Analysis 21
3.1 Arbitrage – Basic run-down. 21
3.2 Arbitrage – The four (main) arbitrage opportunities in CPMM 22
3.2.2 Token A more expensive inside the pool – User has token B: 22
3.2.3 Token A cheaper inside the pool – User has token B:..... 24
3.2.4 Token B more expensive inside the pool: User has token A:..... 25
3.2.5 Token B cheaper inside the pool. 26
3.3 Note on arbitrage opportunities: 27
3.4 Impact of arbitrage on the pool and base CPMM's vulnerability to this 27
3.4.1 Example of arbitrage trade and impermanent loss for LP: 27
3.5 Motivations behind my implementation and solution..... 29
4. System Requirements and Specification 29
4.1 System core aims..... 29
4.2 Functional Requirements..... 30
4.3 Non-Functional Requirements 30
4.4 Key technologies to be used in front-end..... 31
4.3.1 Choice of language and development environment for smart contract development 31
4.3.2 Choice of language and development environment for front-end application development 32
4.3.3 Choice of web technologies for frontend-demo..... 32
5 System Design, Proposed Approach and Implementation 33
5.1 Solidity Fixed point arithmetic 33
5.1.1 Solidity fixed point arithmetic 33
5.1.2 Ethereum 18 decimals 34
5.2 Implement CPMM In Solidity 34
5.2.1 Base CPMM fee mechanism and calculation 35
5.3 Adding Chainlink support for external ratio tracking. 35
5.3.1 Key considerations for fee mechanism 35
5.3.2 Logic behind the fee adjustment during times of arbitrage 36
5.4 Tracking the external ratio over a period or transactions. 36
5.4.1 Recording Simple Moving Average (SMA) of external token ratio: 37
5.5 Rationale behind higher trading fee during times of arbitrage 37
5.6 Variable trading fee calculation..... 38

5.7 Example of variable trading fee use 38
5.8 Note on suitable token pairs for my implementation: 39
5.8.1 Preferable token pairs for the pool 40
6 Testing and Evaluation 40
6.1 Testing the smart contracts. 40
6.2 Evaluation against requirements 41
6.2.1 Smart contract functional requirements 41
6.2.2 Front-end functional requirements 42
6.2 Evaluation 42
6.2.1 Evaluation Metric – Impermanent loss difference between two pools 42
6.4 Evaluation – Scenario setup (trade volume and ratio movement) 43
6.4.1 Modelling trade volume with Poisson Process 43
6.4.2 External price ratio modelling 43
6.4 Evaluation – Constant external ratio 44
6.4.1 Constant external ratio results 45
6.5 External ratio changes by small amount from baseline value 46
6.5.1 Small amount from baseline value results 47
6.6 External ratio changes by large amount from baseline value 48
6.6.1 Large amount from baseline value results 49
6.7 Hyper-Aggressive arbitrage test 50
6.7.1 Hyper-Aggressive arbitrage results 51
6.7 Evaluation findings and Conclusion 52
7 Statement of Ethics 52
7.1 Justification of idea 52
7.2 Legal Considerations 53
7.3 Ethical considerations – Informed Consent 53
7.4 Positive contribution to Society 53
7.5 Ethics (closing) 54
References 54

Abstract

This paper explores the problem of impermanent loss for CPMM liquidity pools and the exact mechanics that allow it to happen. It then explores how smart contracts can leverage real-time off-chain data from Chainlink to dynamically adjust a trading fee in response to external market movements (outside of the liquidity pool itself). The fee mechanism I've implemented aims to

mitigate (not eliminate) the impermanent loss experienced by LPs as while arbitrage does cause a loss in value, its also a necessary mechanic for rebalancing the pools state to match external markets. The evaluation of my mechanism showed a noticeable improvement (50%+) in fee accrual over a large set (thousands) of transactions against the variable fee pool when compared to a regular pool. This improvement in fee accrual while still allowing for profitable arbitrage suggests that off-chain data integration into DeFi applications can be a viable solution in the future.

1. Introduction

1.1 Introduction

This report aims to explore cryptocurrency, decentralized finance (specifically liquidity pools), and some of the algorithms that are used in their various implementations. The report will explore and discuss my software solution (in Section 4). I will then assess the quality of my solution in the context of the cryptocurrency field and discuss any potential shortcomings and improvements that I could make.

1.2 Background

Many people these days flock to centralized exchanges (Binance, Coinbase) to buy various cryptocurrencies such as Ethereum or Bitcoin. While these exchanges are relatively easy to use (especially for those with no background knowledge/experience) there are various associated pitfalls with them. One issue is the trust-based nature of these exchanges. When you deposit funds onto these exchanges, you are 'trusting' that they will not misappropriate your funds with no real guarantees. Once you deposit your money onto one of these centralized exchanges, it is entirely possible (which happens somewhat regularly) for the exchange to stop you from removing them from the platform. Another issue is the large fees that you often incur when using these exchanges. Many of these exchanges charge fees for depositing money, buying/selling assets, and even removing funds from the platform. While these fees may seem small at first, due to their frequency the cost often adds up fast.

This is where Decentralized Finance and Liquidity Pools come into play. Liquidity pools allow for a 'trust-less' exchange of assets, often with much lower fees than those incurred on centralized exchanges.

The importance of this 'trust-less' architecture in relation to the security of user funds has been made clear countless times over the years, but one very well-known case is that of FTX [1]. FTX was a cryptocurrency exchange that gained massive popularity during the 'crypto-boom' [2] of 2021-2022, but subsequently imploded after they were exposed for grossly misappropriating user funds. While the case of FTX is very 'extreme', a fundamental design aspect that allowed this misappropriation to happen was the centralised nature of the exchange. Users of the exchange had no idea what was happening internally and blindly 'trusted' FTX to use their funds as promised.

1.3 Background Key Knowledge

Before reading the rest of this report, it is vital to have a basic understanding of the following terms.

Blockchain

A blockchain is a distributed ledger that is shared across multiple nodes in a decentralized network. This ledger maintains a 'chain' of blocks, each containing transactions that have been verified by the participants (nodes) in the network. Once a block has been added to the blockchain, this history can't be altered. Due to the participation of all network participants, no single entity has 'control' over the ledger, which means no single person can manipulate things without detection. For this reason, blockchain provides the groundwork for the security, transparency, and immutability of data all at the same time.

This makes it a very attractive candidate for finance (De-Fi).

Consensus Mechanism – Proof of Stake

Consensus is the process by which a group of people come to an agreement on some value. In the case of blockchain, these 'people' are the nodes on the network. A consensus algorithm is necessary to ensure that only legitimate transactions are validated and added to the blockchain.

Proof of Stake is the Consensus mechanism that the Ethereum blockchain (and many others) use. Proof of Stake works by choosing validators based on how much of the native token (Ether) they have staked. Validators who don't do their job properly (i.e. byzantine nodes) can have their stake 'slashed', this is where a portion of their stake is permanently taken away from them. By prioritising those with higher stakes while simultaneously having consequences for byzantine nodes, you reduce the chance of random people (with small amounts of Ether) from disrupting the network. Essentially, if you can sabotage the network (have a large stake) you are actively disincentivized from doing this as your stake can be slashed.

Smart Contract (Ethereum Blockchain)

Ethereum smart contracts are essentially programs that are stored on the blockchain. They are (primarily) written in the 'Solidity' programming language, which runs on the EVM (Ethereum Virtual Machine). People write these contracts and deploy them on-chain. People can then interact with the smart contract using its address, and for each interaction (assuming it mutates some state) they will pay some gas fee.

Gas Fees

A gas fee is the 'fee' that you must pay to mutate some data on the blockchain. In the context of the Ethereum blockchain, you will pay gas fees whenever you try to send some Ether to somebody, or whenever you interact with a smart contract that is mutating some state. For example, if you had a Smart Contract that simply had a counter variable, each time you wanted to increment that variable you would need to pay a gas fee as you would be mutating the on-chain state. This gas fee is normally paid in the native token for that chain, in this case the native token of the Ethereum blockchain is Ether.

The gas fee you pay for a transaction depends on two main factors.

1. The current congestion of the network. The busier the network (higher the volume of transactions), the higher the gas fee will be as validators will prioritize transactions of those who are willing to pay higher gas.
2. The intensity of computation that is being performed in the transaction. In the case of our counter contract, the gas fee for incrementing will likely be small.

It is important to note that reading on-chain data is completely free and does not have any associated gas fees. To read data from on-chain, you use an RPC that will then read data from one of the nodes on the network. There are many public-facing free RPC endpoints you can use to read on-chain data.

Token (ERC20)

ERC20 is the standard for fungible tokens on the Ethereum blockchain. ERC20 tokens are not to be mistaken for Ethereum's native token Ether. ERC20 tokens are used interchangeably within the Ethereum ecosystem, for example you might swap some USDC (an ERC20 token) for some USDT (another ERC20 token). Ether on the other hand is used for internal payments within the network such as gas fees.

'Fungible' in this context refers to the idea that two of any given ERC20 token are functionally indistinguishable, in the same way that a regular £5 note is interchangeable with any other £5 note.

The ERC20 standard came about because of the rather chaotic nature of token creation beforehand. Anybody could create tokens (with a Smart Contract), but since there was no standard, there was no guarantee that everybody could easily interact with these tokens (i.e. sending / receiving).

The standard itself is a list of functions and events that **must** be implemented on a tokens smart contract for it to be classified as an ERC20. These are listed below:

- **TotalSupply**: The total number of tokens that will ever be issued
- **BalanceOf**: The account balance of a token owner's account
- **Transfer**: Automatically executes transfers of a specified number of tokens to a specified address for transactions using the token
- **TransferFrom**: Automatically executes transfers of a specified number of tokens from a specified address using the token
- **Approve**: Allows a spender to withdraw a set number of tokens from a specified account, up to a specific amount
- **Allowance**: Returns a set number of tokens from a spender to the owner

For any ERC20 token, you should be able to call these methods / events.

Minting / Burning Tokens

Minting a token is essentially 'creating' another one of the tokens. Remember that since ERC-20 tokens are fungible, there is no detectable difference between two of the same minted ERC20 tokens.

'Burning' a token refers to taking that token out of circulation. When we burn tokens, they aren't literally deleted from existence, they are instead typically sent to an inaccessible wallet address.

De-Fi

De-fi is an umbrella term that encompasses various financial applications that are built on the blockchain. The main 'aim' of most De-Fi applications is to remove the 'central' authority and/or middlemen in whatever financial tool they are implementing. For example, liquidity pools implement an 'exchange' without the requirement of a central broker to oversee/manage the selling/buying of assets. Additionally

Crypto wallet (wallet address)

To send/receive cryptocurrency assets (tokens), you must have a wallet. The assets themselves are stored on the blockchain, but a user's ability to send/receive is determined by the public/private key pair for their address. The public key is used by other people to send assets to you, and your private key is used to approve transactions on your behalf (i.e. send your assets to someone else).

Centralized Exchange (CeX)

A centralized exchange is an asset exchange that allows people to buy/sell assets at their desired price. Examples of centralized exchanges in the cryptocurrency space are Binance and Coinbase. Both exchanges work by giving you a wallet address, you can then buy/sell tokens using that wallet address to send/receive. The main issue with centralized exchanges is that your private keys are managed by the exchange, not you (the owner). While you have access to the private key, so does the exchange, meaning that theoretically, they could steal all your assets.

CeX's facilitate trades using the order book model. This is where somebody lists a token for sale at price X, and that order is only fulfilled when another person wants to buy that token for the same price X.

De-centralized exchange (DeX)

DeX's aim to provide the same service as CeX's but without the need for the central broker (the exchange). In the case of the Ethereum blockchain, these are implemented with smart contracts. Instead of having a central broker who facilitates the buying and selling of assets, you have a smart contract that has all the logic pre-programmed. Users can interact with this smart contract by either calling functions directly through an RPC, or most often through their front-end websites. Popular examples of DeX's are pools like Uniswap and Pancakeswap.

Due to the fundamental difference in the implementation, most exchanges do not follow the traditional CeX 'order book' approach, and instead tend to implement **Liquidity pools.**

Liquidity Pool

1.4 Project Description

This project aims to take an existing concept (the liquidity pool) and attempt to address one of the core issues that the pool architecture suffers from (Impermanent loss). The implementation for this project includes a set of backend 'smart contracts' written in the Solidity programming language, as well as an interactive front-end written with JavaScript and React which the user can then use to interact with said smart contracts.

This report will detail relevant background knowledge concerning blockchain, cryptocurrency, tokens, DeFi and liquidity pools. It will then detail various algorithms that are used for liquidity pools, their strengths and weaknesses, and then finally my approach to improving one of them.

This algorithm with the proposed improvements will then be implemented in Solidity, and then an interactive front-end will be implemented for people to interact with the smart contracts.

1.5 Project Aims and Objectives

This section includes details about the different aims of my project and the steps I plan to take to achieve these aims. These aims will then be assessed later in the report.

Step 1: Research different liquidity pool algorithms and general cryptocurrency knowledge

For me to form a solid baseline for my project, it is important for me to first research relevant background information/literature. In my case, this includes gaining a solid understanding of the cryptocurrency space. More specifically, I must gain a solid understanding of different liquidity pool algorithms, how they work, and their strengths and weaknesses. This will give me a solid foundation to work from when developing a solution.

Step 2: Research possible improvement(s) for a liquidity pool algorithm

After researching relevant liquidity pool algorithms and existing liquidity algorithms in the cryptocurrency space, I then need to choose an approach for my project. I have opted to go with integrating Chainlink into an existing algorithm, to address impermanent loss.

Step 3: Write smart contract code to implement algorithm with Chainlink integration and other adjustments/optimisations

Once I have decided on my approach, I need to write the actual smart contract logic.

Step 4: Perform testing with a set of example transactions to assess the effectiveness of my solution.

Once the smart contract has been implemented with the liquidity pool algorithm and Chainlink integration, I need to assess its effectiveness. To do this, I plan to execute a large batch of transactions against both my solution and the original 'unaltered' algorithm. I can then analyse the results of the transactions on both versions and draw conclusions from the outcome.

1.6 Possible Issues

Due to the nature in which Ethereum transactions are processed, and the associated costs that come with it, I am doing all my development and testing on one of the Ethereum test nets (Sepolia testnet). For this reason, there may be some restraints on exactly what I am able to do given that I am not on the mainnet. If testing contracts on testnet is too slow due to transaction times I will have to find an alternative.

1.7 Summary

In summary, this project will mainly focus on liquidity pool algorithms and analyse their strengths/weaknesses which are inherent given the nature of the underlying technology. My solution aims to try and 'bridge' this gap by integrating an existing technology (Chainlink) into one of these algorithms along with some adjustments to address the issue of Impermanent loss. These things will be discussed in more detail in later sections.

2. Literature Review

This part of the report will discuss some existing solutions/products in the cryptocurrency / De-Fi space that perform similar functions to that of my proposed solution. It will include a discussion about various background topics that are important to understand in the context of my report, as well as various existing solutions that I have deemed relevant to my project. Note that when talking about blockchain I am specifically talking in the context of the Ethereum blockchain.

2.1 Methods to Buy Cryptocurrencies.

2.1.1 Centralized Exchanges

The traditional method by which people purchase cryptocurrencies is through regular 'centralized exchanges. The main reasons for their popularity are the ease of access to inexperienced/retail users, and the tendency to be similar in design to a regular brokerage application [3]. This 'familiar' design, paired with the often highly intuitive usage of their applications tends to make for a very smooth experience.

Despite this, there are some very serious drawbacks to centralized exchanges, especially in the context of cryptocurrency. One of the inherent traits of cryptocurrencies is their decentralized nature. Using centralized exchanges to facilitate the buying/selling of decentralized assets is somewhat paradoxical, and you can argue that it goes against the nature of these assets. Exchanges such as Binance and Coinbase are examples of centralized exchanges, when you deposit money onto these exchanges you are forfeiting control of your assets and 'trusting' that they will do right by you (not trust-less).

Another drawback of CeX's is the government regulation that invariably comes with using them. Binance and Coinbase require users to submit government ID in accordance with KYC (Know Your Customer) regulations. These regulations are in place to fight against things like money laundering and fraud, by forcing people to tie their identity to their assets. While you could argue

that most people don't need to be concerned with things like KYC, it does break all forms of anonymity as your identity is now tied to your crypto assets. KYC can also be a genuine concern for law abiding citizens in more authoritarian countries such as China.

2.1.2 DeX's (Decentralized Exchanges)

Decentralized exchanges aim to address many of the inherent and unavoidable drawbacks of centralized exchanges. The only requirement to use a DeX is a wallet that supports the desired tokens and the user connecting said wallet to the DeX. There is no requirement for KYC (know your customer), and due to their decentralized nature, there is no overseeing authority that can be a bad actor.

2.2 DeX Implementations (Liquidity Pools)

2.2.1 Order book model – Why it doesn't work with blockchain.

As stated in the introduction, DeX's tend to have a fundamentally different design pattern to that of regular CeX's. CeX's tend to use the 'order book' model where buy/sell orders are directly matched based on some agreed price. While this 'order book' model is fine in the context of CeX's, it is neither appropriate for blockchain implementations, nor the most optimal.

One of the main drawbacks of this order book model when applied to blockchain is the requirement for storage and the resultant cost. Blockchain storage is extremely expensive due to its distributed nature. When you want something stored on-chain, that thing is stored across multiple nodes. The size of an order book grows with the amount of people trying to buy/sell, that means in times of high traffic the order book on-chain could become so large that associated gas fees would become too high.

It is clear from the explanation above that the order book model from CeX's is fundamentally incompatible with blockchain due to its distributed nature and high cost of storage. For this reason, DeX's tend employ a different model known as the 'Liquidity pool'.

2.2.2 The Liquidity Pool approach

A liquidity pool in its simplest form is a pool (reserve) of two tokens (token A and token B). The tokens in the reserves are supplied by liquidity providers (LPs). Users of the pool can then come to the pool with an amount of token A / token B and swap for the other. Liquidity providers are incentivized to provide liquidity through the accruement of trading fees. Each time somebody executes a swap between token A and B, a small percentage fee is taken and distributed to liquidity providers.

This approach fundamentally differs from the order book approach as the swaps (buying / selling between token A and B) are facilitated purely by the liquidity providers, rather than the person participating in the swap. In an order book (I.e. a regular CeX), a buy order will only execute when there is a different person who is selling the asset for the same price.

With the liquidity pool approach, assuming there is sufficient liquidity in the pool (provided by LPs), anybody can come to the pool with an amount of one token and swap for the other, without the need for a person on the other end.

2.2.2.1 Liquidity Provision

As stated above, for swaps to take place within the pool, liquidity needs to be provisioned from liquidity providers. Liquidity providers provide liquidity (amounts of token A and/or token B) to the pool, in return for trading fees. Their liquidity is then used to facilitate the swapping between token A / token B for users of the pool.

When users deposit liquidity to the pool, they are compensated with LP (Liquidity Pool) tokens that entitle them to a proportional of the pool's reserves. For example, if there is only one liquidity provider to the pool, then that providers LP tokens will entitle them to all the reserves. If there are two liquidity providers who have each provided equal amounts of liquidity, then their LP tokens will each entitle them half of the reserves.

2.2.2.2 Burning LP Tokens

As stated above, liquidity providers are issued with liquidity pool (LP) tokens when they add liquidity. Generally, this is in the form of another ERC-20 token that the liquidity pool contract can mint and burn. LP tokens are minted and issued to liquidity providers when they provide liquidity to the pool and are then burned when the provider wants their liquidity back.

2.3 Automated Market Makers (AMMs) and existing algorithms for implementations

Automated market makers are a type of DeX where a balancing algorithm is used to decide the internal exchange rate of the pool, and to facilitate trades (swaps) between the two tokens. They are 'automated' in the sense that one person can buy or sell tokens to the pool without the need for a corresponding buy/sell order. People trade directly with the algorithm itself, rather than with other traders.

There are various existing algorithms for AMMs, but the two that have seen the most use are the CSMM and CPMM algorithms.

2.3.1 Constant Sum Market Maker (CSMM)

CSMM is an example of an automated market maker algorithm that did not see widespread adoption due to a severe issue with the way the algorithm balances the quantity of the two tokens.

The equation for this algorithm is $x + y = k$ where:

- x is the quantity of token A in the pool
- y is the quantity of token B in the pool
- k is the 'constant sum' that needs to be maintained for a given amount of liquidity

(Note that when liquidity is added (i.e. another LP comes along to add liquidity), then k is adjusted accordingly).

The idea of this algorithm is that when users do swaps between token A and token B, if you drain one reserve, the other reserve must increase by the same amount to keep the constant k (i.e. amount of token A in = amount of token B out).

For example, if we had 100 token A and 100 token B (k = 200) and we wanted to get 50 token B out, then we would have to 'pay' 50 token A to keep $k = 200$. The algorithm is very simple to implement and to understand but has a severe vulnerability that makes the algorithm essentially useless for real use in a liquidity pool.

This is the fact that due to the linear nature of the equation; the swap ratio of the pool does not change to prevent one of the reserves from being drained. Draining one of the reserves is when the balance of token A or token B within the pool approaches zero, effectively making the pool useless as there is no liquidity to facilitate swaps.

An example of how the reserves might be drained is as follows:

Example of reserve draining

Token A balance: 100.

Token B balance: 100.

$$100A + 100B = 200$$

A user can come to this pool and swap 100 of token A for 100 of token B. After this, there will be 200 token A, and 0 token B in the pool i.e.

$$200A + 0B = 200$$

The AMM is working as intended but has very easily been completely drained of its entire token B supply.

Given that users are required to provide equal values of liquidity in both tokens (i.e. equal values of token A and token B), they will reasonably expect to get back roughly the same amount of each token that they put in. If one of the reserves get drained and the LP tries to remove their liquidity, they will instead receive only tokens from one reserve, which is problematic since LPs expect to get back at-least what they put in.

As demonstrated above, the CSMM algorithm has some severe issues with balancing reserves and ensuring that neither reserve gets drained down to zero. This is a fatal flaw and makes the algorithm essentially obsolete in practice since an arbitrageur can come along, drain the reserve of one token and there is no incentive / reason for anybody to try and remedy the issue.

2.3.2 Constant Product Automated Market Maker (CPMM)

The CPMM algorithm addresses the issue of reserve draining by using a different formula that adjusts the exchange rate depending on the ratio of the two reserves in the pool. The formula for this algorithm is:

$$x * y = k$$

- x is the quantity of token A in the pool
- y is the quantity of token B in the pool
- k is the 'constant product' that needs to be maintained for a given amount of liquidity

The fundamental idea behind this algorithm is that as the quantity of token A in the pool decreases, the swap ratio adjusts to make token A more expensive (and visa versa). This is a very big difference from the CSMM algorithm where the swap ratio was fixed, meaning there was no 'balancing' mechanism to try and keep the reserves relatively stable and filled.

The initial swap ratio (price) of the CPMM is decided by the initial deposit of liquidity into the pool. For example:

Initial Token A deposit: 100

Initial Token B deposit: 200

$$100 * 200 = 20{,}000$$

With this initial deposit, we dictate that token A = two token B i.e. $A = 2B$

2.3.2.1 CPMM Adding Liquidity

Per above, the 'price' of the tokens in the pool is decided by the ratio of the two. Since the price is decided by the ratio of the two tokens, when new LPs come to add liquidity to the pool, we need to ensure that their added liquidity does not cause a change in the price of the pool.

To ensure this, we enforce a rule that the ratio of the tokens being added matches that of the pools current reserve state. Without this rule, anybody could manipulate the pool price by adding specific ratios of one of the two tokens.

For example, if the current ratio of token A: token B is 1:10, liquidity providers will have to match this ratio in their deposit.

2.3.2.2 Swaps

Unlike the CSMM, the CPMM swap ratio is not linear, i.e. if we put in 100 of token A, we do not necessarily get 100 token B out. Instead, the amount that we get out is dependent on the current ratio of the tokens in the pool, and the size of the trade relative to the size of the reserves. The smaller the 'size' of your trade is relative to the size of the reserves, the more 'linear' the trade execution will appear.

2.3.2.3 CPMM Swap Ratios breakdown

Despite the relative simplicity of the CPMM formulae, understanding how swap ratios and liquidity provision work can be harder than it may initially seem. I will now break down how to calculate swap ratios and why it is very important to understand in practice.

<u>Calculating swap ratios</u>

We have the initial formula $x * y = k$ where x is quantity of token A, y is the quantity of token B, and k is the constant that must remain the same for a given amount of liquidity in the pool.

Say we have a pool with 100 token A and 200 token B: $100 * 200 = 20000$.

A trader comes to our pool and wants to swap 20 token A for an amount of token B, how do we calculate the amount of token B to send back?

$(x + dx) + (y - dy) = k$ where dx is the amount of token A coming in, and dy is the amount of token B being sent back to the trader. If we solve for dy (the amount of token B in exchange for our amount of token A), we get:

$$dy = \frac{ydx}{x + dx}$$

Note: to calculate dx simply swap x/y and dx/dy.

From this formula, we can see that the amount of token B returned to the trader (dy) depends on the remaining amount of token A in the reserves and the amount of token B that you are inputting. There are two main cases to consider here:

1. dx is small compared to x: If dx (the amount of token A coming in) is relatively small compared to x (the reserve amount of token A in the pool) then the effect on the division will be negligible. In this case, the number of tokens returned will be roughly in line with the ratio of the tokens in the pool since the amount of token A coming in is negligible compared to the size of the reserves.
2. dx is non-negligible compared to x: In this case, $x + dx$ will be much larger than dx meaning that the outcome of the division will be much smaller so you will get less tokens out. This situation occurs when the amount of the token coming in is a large portion of the existing reserves. There are two main realistic scenarios where this occurs:
    a. The pool has a low amount of liquidity, in this case people will only get a 'fair' exchange rate on very small trades.
    b. You request a trade that is unusually large compared to the size of the reserves. In practice, most liquidity pools end up with millions or billions of dollars' worth of tokens, so this is rarely an issue with popular liquidity pools with a lot of liquidity providers.

For the above reasons, it is extremely important for the functionality and attraction of the pool that there is enough liquidity so that large trades can take place without offering poor exchange rates.

<u>Example swap with low liquidity</u>

Say we have a pool with 100 token A and 200 token B: $100 * 200 = 20000$

With the above pool state, we can assume that token A is twice as valuable as token B i.e.:

$$A = 2B$$

A trader might come to this pool with 50 token B and expect to get 25 token A back (from above), but if the amount of liquidity in the pool is small relative to the trade they are making then the exchange rate can be quite bad.

With this current pool state, the trader would get the following amount of token A for 50 token B:

$$dx = \frac{xdy}{y + dy}$$

$$= \frac{(100 * 50)}{200 + 50} == 20$$

So due to the low liquidity of the pool, the trader has received 20 token B when they were reasonably expecting somewhere around 25.

Example swaps with high liquidity.

Suppose a trader goes to use the pool when there is a large amount of liquidity in the pool:

100000 token A, 200000 token B, k = 20,000,000,000

If the trader does the same trade, then we instead have:

$$dx = \frac{100{,}000(50)}{200000 + 50} = 24.994$$

In this trade due to the high amount of liquidity in the pool, we received a much better exchange rate getting very close to 25 (the assumed price given the state of the pool).

2.3.3 CPMM and the need and utility of arbitrageurs.

Arbitrage is when you buy an asset from one market and then sell it at a different market for a better price. The difference in price between the two markets allows you to make profit from these trades.

2.3.3.1 Arbitrageurs

Suppose the price of Ethereum (a cryptocurrency) was £1000 on Binance but £1100 on Coinbase, arbitrageurs would take advantage of this situation. They would keep buying Ethereum from Binance until the price matches the price on Coinbase (£1100). They would then take this Ethereum to Coinbase and sell it all for £1100. Since they have sold it at a higher price than they bought at, after selling the ETH to Coinbase they will be left with some profit.

2.3.3.2 Relevance of Arbitrage to CPMM algorithm

Arbitrageurs are a vital participant in many AMM's, especially in the CPMM algorithm. Whenever a trade is executed in our AMM, the composition (ratio) of the reserve's changes, thereby changing the price.

Arbitrageurs will notice this price change (relative to external markets) and take advantage of it by either buying one of the tokens for cheaper than external markets, or by selling a token to the pool for an artificially higher price.

This arbitrage process is beneficial (to an extent) for everyone involved.

- LP's earn trading fees from each trade that happens in the AMM whether it's an arbitrageur or someone just wanting to do regular swaps.
- Arbitrageurs earn profit by taking advantage of the price difference in different markets. By doing arbitrage trades, they provide trading fees.
- Regular users of the AMM benefit as this arbitrage keeps the relative prices of the tokens in line with external markets.

2.4 Impermanent Loss

Impermanent loss refers to the 'loss' that can occur due to the difference in the price of assets you provide to a liquidity pool at the time of deposit vs time of withdrawal.

The more volatile the assets in a liquidity pool are, the higher the risk of impermanent loss. This is because impermanent loss is caused by the pool rebalancing to account for this external price change.

Impermanent loss is called 'impermanent' because the loss is not actually realised by the LP until they withdraw their liquidity. It is possible for your investment to lose value for a period (due to price changes), but you only 'realise' the loss if you withdraw your tokens at this point. The pool can return to the 'initial' state if external prices return to what they were at the time of deposit though this is not something that should be assumed to happen.

2.4.1 Impermanent Loss rough example / explanation

Assume we have a liquidity pool with the asset pair ETH / USDC. A liquidity provider deposits liquidity at the ratio 1 ETH / 1000USDC. After depositing their liquidity, the price of Ethereum rises to $1200 on external markets (i.e. Binance / Coinbase).

As described before, arbitrageurs will take advantage of the price difference and buy cheap ETH from our pool and sell it to the external markets for profit until the pool price (token ratio) changes to match that new external market price. The pool price adjusting to match external markets involves the token ratios changing which will affect the value of the LPs share of the reserves.

When the LP comes along and removes their liquidity, they will receive their percentage share of the ETH and USDC that is in the pool. There is a chance that the value of the ETH and USDC they receive will be lower than if they simply held their tokens instead of investing them in the pool.

2.4.2 Appropriate token pairs for liquidity pools

To recap, liquidity pools are pairs of tokens that are 'pooled' to facilitate the swapping between token A and token B. As discussed above, using the CPMM algorithm with volatile asset pairs is problematic as asset price changes on external markets cause the pool composition to change. This composition change results in value being siphoned away by arbitrageurs, resulting in LPs potentially losing value on their investment.

For the above reasons, a lot of liquidity pools use 'Stablecoin' pairs though it is strictly necessary.

What are Stablecoins?

Stablecoins are cryptocurrencies where the value of the asset is pegged to some other asset. There are two general types of stablecoin, algorithmic and reserve backed.

Reserve backed stablecoins operate under the idea that there is a real life backing for every unit of that currency. For example, USDC is a cryptocurrency that has $1 in a vault for each USDC token that is minted. The real-life fiat currency is used as collateral for the crypto.

The other type of stablecoin is algorithmic stablecoins. Algorithmic stablecoins do not necessarily inherit their value from collateral reserves, they instead maintain their value by manipulating supply using special formulae.

The important thing to understand is that Stablecoins when implemented properly and legitimately maintain their value and are extremely 'stable'. For this reason, they make great candidates for things such as liquidity pools as the lack of value change stops the pool composition from shifting around constantly.

2.4 Smart Contracts (Solidity) and Security for Ethereum Blockchain

Smart contracts are programs that are deployed to the blockchain. On the Ethereum Blockchain, Solidity is the language of choice for this.

As discussed previously, smart contracts are effectively the substitute for the marketplace. User funds (from liquidity providers) are stored in the smart contract, and therefore ensuring the security and trustworthy design of the smart contract is extremely important.

For smart contract developers it is extremely important to properly consider the security of the contract. For those who are considering interacting with smart contract(s) (such as liquidity pools) it is also extremely important that they understand exactly what they are interacting with.

2.4.1 Smart contract ownership / permissions.

Variable / Function permissions

One of the most important things to ensure when writing a smart contract is that permissions are restricted properly on the contacts various methods.

Smart contract code is very similar to a regular object-oriented language, with public/private variables and functions. Things marked with public are viewable/callable by anybody while things marked as private are only accessible and callable by the contract itself (i.e. internal functions). If a core internal function is left public for whatever reason, all the funds in the pool could be at risk of being stolen.

Ownership of the contract

Ownership is another concept that is integral to the security of smart contracts. While Solidity has inbuilt functionality for public/private variables and functions (like most programming languages), ownership is another concept to tackle.

There are functions that need to be publicly accessible (i.e. to entities other than the contract itself), but only to certain people (wallet addresses). In this situation, ownership restrictions need to be implemented.

Solidity allows for restrictions like this by using modifiers on functions. These are essentially checks that are ran before a function can be run. Given the make-or-break nature of smart contract security, it makes sense to use verified and trusted solutions for this rather than trying

to re-invent it from scratch. OpenZeppelin provides an open-source framework for implementing secure smart contracts. Since they're open source they are publicly verifiable and given the extremely high rate of adoption you can reasonably assume that their solutions work.

OpenZeppelin provide a smart contract for Ownership that allows you to restrict given functions to only certain wallet addresses which is vital to the functionality of the liquidity. Namely, only the liquidity pool (its deployment address) should be allowed to mint / burn LP tokens, along with other restrictions.

2.5 Addressing Impermanent Loss

As discussed above, impermanent loss is an inherent and unavoidable risk when using the CPMM algorithm for liquidity pools with tokens that have any level of volatility.

The aim of my implementation / solution is to attempt to mitigate this impermanent loss with the assistance of external data using Chainlink.

The AMM algorithms I have covered in this literature review (CSMM and CPMM) are both isolated in the sense that they don't rely on any kind of external data to operate. In some ways this isolation is a strength as they're entirely self-contained and not vulnerable to erroneous incoming data, but this isolation also brings weaknesses.

My aim it so integrate Chainlink into the CPMM algorithm to make it more reactive to external markets, thereby mitigating the impermanent loss that arbitrageurs cause.

2.6 Oracles for off-chain data aggregation

Blockchains are inherently isolated, this is part of their design. They do not have an inherent ability to read in off-chain data for security reasons. Blockchains and their internal operations are designed to be fully isolated so that there are as few vulnerabilities as possible.

While this makes them secure, it also puts a very hard limit on the applicational ability of smart contracts. There are many applications that would benefit from the security guarantees that blockchain provides but since there is no in-built access to external data, the core of the applications is not possible to implement.

To integrate off-chain data into on-chain smart contracts, we use oracles.

2.6.1 What are Oracles?

Oracles are secure pieces of middleware that facilitate communication between a given blockchain and an off-chain system (normally in the form of a data source). There are two main components to oracles.

<u>Off-chain component</u>

The main purposes of the off-chain component of an oracle are to fetch off-chain data, do some off-chain processing and format the data appropriately. After this, the data is then sent to the on-chain component.

<u>On-chain component</u>

The on-chain component of an oracle is responsible for serving the off-chain data to on-chain consumers. The on-chain smart contract will have functions that can be called for data providers to send the off-chain data to. Consumers of the off-chain data will generally call this

smart contract which will then return the formatted off-chain data that has been securely provided to itself.

2.6.2 Off-chain data integration example

An example of an application that could utilise blockchain technology very well but is limited by off-chain data reliance is insurance. Policy holders would pay their insurance money into a smart contract (likely in the form of some stablecoin), then some off-chain event would trigger insurance payouts for the policy holder.

The incentive for storing funds on-chain and having events triggered on-chain is that smart contracts can be made open source and are audited. People would be able to verify the integrity of the insurance policy they are taking out and verify its authenticity / safety.

2.6.3 Why centralized oracles do not work.

The reason that blockchains don't have inbuilt access to off-chain data is the fact that by relying on one unverified API for data, it makes contracts nondeterministic. Using a single source for a given piece of information introduces a single point of failure to the determinism of the smart contracts execution which effectively nullifies the entire point of the underlying decentralized architecture of blockchain.

A centralized oracle is vulnerable to many kinds of attacks and manipulation that make it non-viable for usage by blockchains. Vulnerabilities include DDOS attacks, incompetence and general outages, all of which are unacceptable for our use case.

2.7 Chainlink for secure off-chain data aggregation

As discussed above, centralized oracles are not a viable use case for off-chain data access due to the single point of failure they introduce which breaks the deterministic nature of blockchain.

Chainlink addresses this issue by providing decentralized oracle networks (DONs).

- Chainlink is open source, meaning that the blockchain community can verify the security and reliability of the Chainlink source code and functions.
- Chainlink implements decentralization at the data gathering and node level. This addresses the issue of single point of failure that centralized oracles suffer from.
    - This decentralization helps guarantee timeliness, availability and resistance to manipulation for the data.
- Nodes in a DON sign the data that they have fetched and provided to smart contracts. This allows users to identify reliable and unreliable nodes based on their performance history.

2.8 Use of Chainlink for my implementation and how to interact with Chainlink.

I will be using Chainlink oracles in my implementation for fetching off-chain prices of the assets within the pool. This will give my on-chain smart contract implementation access to secure off-chain data that I can then use to influence on-chain decisions.

2.9 Liquidity Locking in liquidity pools.

Liquidity locking in liquidity pools is when you have a minimum period after a liquidity deposit before the liquidity provider can remove their liquidity from the pool. This is a very common practice in liquidity pools (regardless of which algorithm the pool is using). The main reason for liquidity locking is that it helps to keep the price more stable.

As demonstrated earlier in this literature review, in the CPMM algorithm, the exchange rate of tA / tB is non-linear and dependent on both the **quantity and ratio** of the two tokens in the pool. As a result, LPs adding/removing liquidity can have an impact on the exchange rate (especially for larger trades). By enforcing a minimum period of liquidity provision, the price is less likely to fluctuate due to people adding/removing liquidity constantly.

Liquidity locking is enforced at the smart contract level. A common implementation for this is to maintain a mapping within the contract of wallet address -> deposit timestamp. This mapping is updated whenever an LP deposits liquidity, then when they come to withdraw their liquidity, the mapping is checked. While this mechanism is simple to implement and quite effective, the mapping will continue to grow as more people deposit liquidity which will affect gas fees as storage space requirements increase.

3. Problem Analysis

In this section of the report, I am going to conduct an in-depth analysis of the exact issues that are apparent with the existing CPMM algorithm, and then how my solution aims to address these issues.

3.1 Arbitrage – Basic run-down.

Arbitrage is the buying and selling of items in different markets to exploit price differences between them. A simple example of an 'arbitrage opportunity' would be if you found two stores selling apples. Store A sells apples for 50p, while Store B sells them for 70p. An 'arbitrageur' could exploit the price difference by buying apples from Store A (50p) and selling them to store B (70p), pocketing 20p in profit per apple.

In the context of the CPMM (Constant Product Automated Market Maker), arbitrage opportunities can constantly arise due to the small price movements that occur during each trade.

3.2 Arbitrage – The four (main) arbitrage opportunities in CPMM

When using the CPMM algorithm for our liquidity pool, there are four different arbitrage opportunities that can arise during its operation. Each arbitrage opportunity arises because of a combination of external market price changes, and price impact because of swaps within the pool.

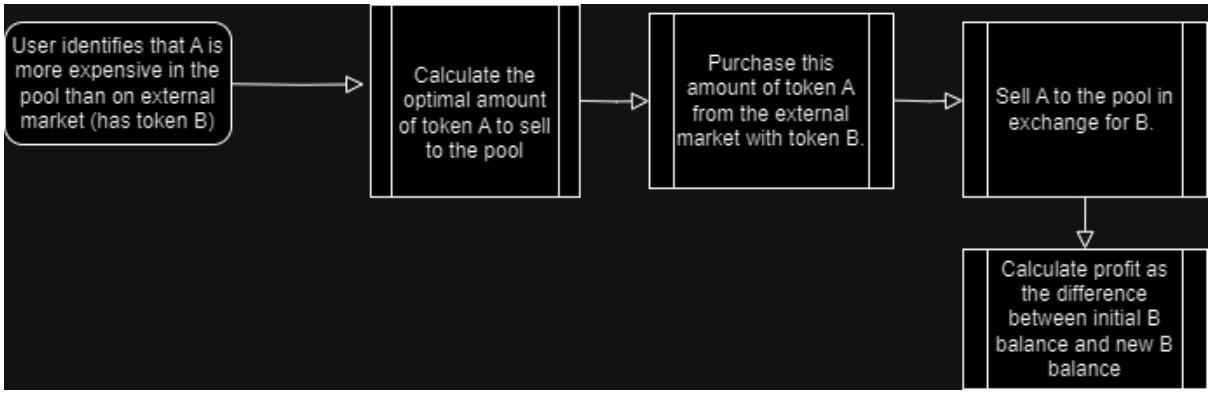
It's important to note that these arbitrage situations are done under the assumption that this external market is some CeX (Binance / Coinbase) that has ample amounts of token A and token B for sale such that individual trades have no price impact, and that this CeX allows users to swap A for B and B for A.

1. Token A more expensive inside the pool than on external markets.
    a. The price of A inside the pool is dependent on the ratio of A:B inside the pool. For example, if the ratio B:A in the pool is 2:1 but 1. The pool price is impacted by people swapping between token A and B, which results in the reserve ratios changing. As such, the pool price of A can increase because of people swapping large amounts of B for A from the pool.
    On the other hand, the price of A and B can change on external markets independently from our pool. For example, one token A might cost 2 token B inside our pool, while it only costs 1.5 token B on an external market. This creates an arbitrage opportunity where people can buy token A externally (with token B) for cheap and sell it to the pool for more token B than they originally spent (profit).
2. Token A is cheaper inside the pool than on external markets.
    a. In this situation, arbitrageurs can buy token A from the pool for cheap, and then sell it to external markets. More specifically, they keep buying A from the pool until the pool price for A (in terms of B) rises to match that of the external market.
3. Token B more expensive inside the pool than on external markets.
    a. If Token A is cheaper inside the pool, then logically Token B must be more expensive (as the price is just the ratio A:B). In this case, arbitrageurs are incentivized to sell token B to the pool (for A) until the price of B falls to match the external market.
4. Token B cheaper inside the pool than on external markets.
    a. This arbitrage opportunity occurs at the same time as opportunity 1 above. If token A is more expensive inside the pool, then B is cheaper. Arbitrageurs are then incentivized to buy B from the pool (for A) until B's price rises to match external markets.

The key thing to understand about arbitrage is that it is possible whenever there is a price difference in two separate markets.

3.2.2 Token A more expensive inside the pool – User has token B:

In the following sections I will break down the four different arbitrage opportunities on a more technical level including the specific calculations arbitrageurs use to take advantage of price differences.



The first opportunity that arbitrageurs will take advantage of is when token A is more expensive inside the pool, and therefore cheaper on external markets. This situation arises due to either the pools composition changing (causing the token price to increase), or the token price decreasing on an external market (meaning that the pool price is higher in comparison.

The above flow diagrams show the steps that an arbitrageur will execute to execute the 'optimal trade' against the pool. As explained earlier, each swap in the pool causes the token ratio to change, and therefore the price (this is called price impact). The 'optimal' swap is the swap that brings the external price (ratio) inline with the pool price (or ratio). More specifically, the optimal swap here is to sell enough token A to the pool such that the pool price falls to match the external market price (due to the increased supply of token A it receives).

3.2.2.1 Formulae breakdown

We know that the core formulae for the CPMM algorithm is $x * y = k$ where:

- x is the quantity of token A inside the pool
- x is the quantity of token B
- k is the product of X and Y which must remain constant for any given amount of liquidity in the pool.
- Given this arbitrage situation we also have our external market price P which is equal to y/x (the ratio of y to x) <u>on the external market</u>.
- The 'pool price' is simply the ratio of y/x in our pool.

In this arbitrage situation, we are selling token A to the pool. To solve for the 'optimal trade' we want to find the value for x that brings the pool ratio (y/x) to equal P (external market ratio).

<u>Calculations</u>

We define two variables:

1. $x\Delta \ = optimal\ amount\ of\ X\ to\ sell\ to\ the\ pool$
2. $x_{new} = x_0 + x\Delta$

From the base formula $x * y = k$ we know that $y = \frac{k}{x} or\ in\ our\ case\ y = \frac{k}{x_{new}}$

We know that P (external market price) is equal to the ratio y/x, so we can write this as:

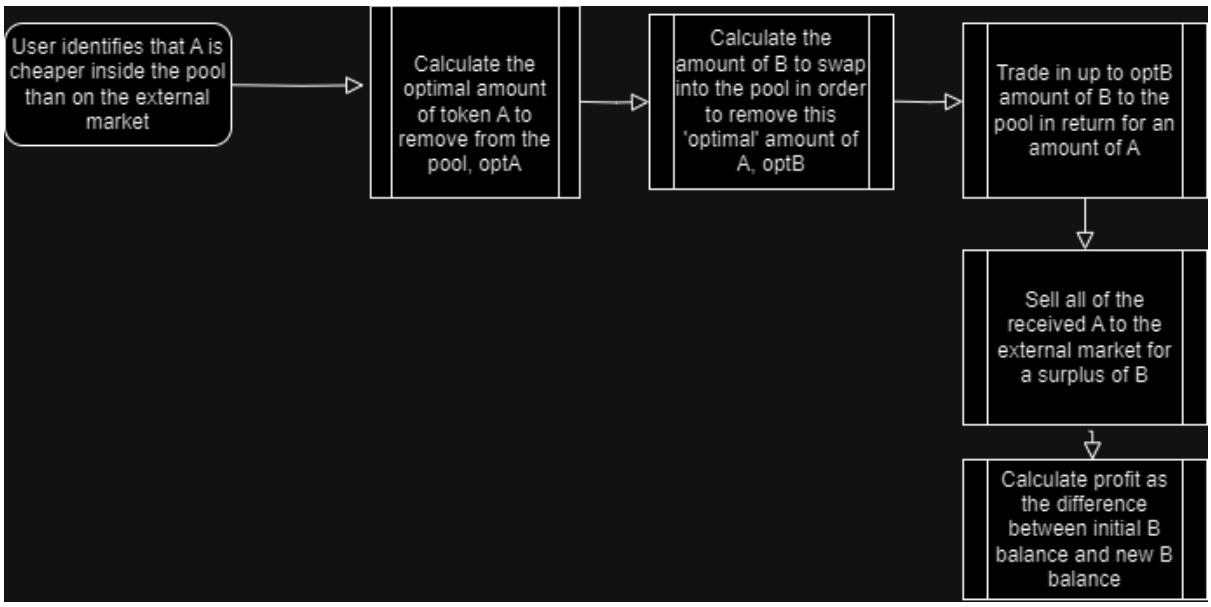
$$\frac{y}{x} = P\ or\ \frac{\left(\frac{k}{x_{new}}\right)}{x_{new}} = P$$

We can simplify this equation to solve for $x\Delta$ and get the value:

$$x\Delta = \sqrt{\frac{k}{p}} - x_0$$

This value for $x\Delta$ is the optimal / maximum value that should be swapped into the pool to most efficiently exploit the price difference (bring the pool price for A down to match the external market). With this value, arbitrageurs can purchase up to $x\Delta$ amount of token A from the external market for y_0 token B, then sell the token A to the pool in return for y_1 amount of token B. The difference $y_1 - y_0$ is the arbitrage profit from this trade.

3.2.3 Token A cheaper inside the pool – User has token B:



This opportunity arises when the external market price falls below the pool price, of if the pools reserve composition shifts such that the pool ratio falls below external markets.

In this arbitrage situation the goal is to buy enough token A from the pool (cheap) until the pool price raises to match the external ratio. Essentially, we want to solve for the optimal amount of token A to remove from the pool (rather than optimal amount to add).

<u>Calculations</u>

We define two variables:

1. $x\Delta = optimal\ amount\ of\ X\ to\ buy\ (remove) from\ the\ pool$
2. $x_{new} = x_0 - x\Delta$

We solve for $x\Delta$ the same way we solved in the first arbitrage opportunity, however this time the final rearrangement is slightly different due to the different x_{new} definition. We end up with a value of:

$$x\Delta = x_0 - \sqrt{\frac{k}{p}}$$

This $x\Delta$ value is the optimal amount of x that we should **<u>REMOVE</u>** from the pool to raise the pool price to meet the external price. Now that we know the optimal amount of X to purchase, we now need to calculate the quantity of y that we need to swap into the pool to get $x\Delta$ out.

When purchasing from the external market we can simply use the given market price / ratio and multiply / divide appropriately as we are assuming trades have no price impact, the same is not true for our pool. Therefore, we need to calculate our value of y_{new}.

We know from the constant product rule that for any amount of provisioned liquidity in our pool:

$$x * y = k$$

Since we have $x\Delta$ (optimal amount to remove), we can calculate the new reserve state of x after the trade to be $x_{new} = x_0 - x\Delta$. With this information, we can solve for the new reserve state y_{new}.

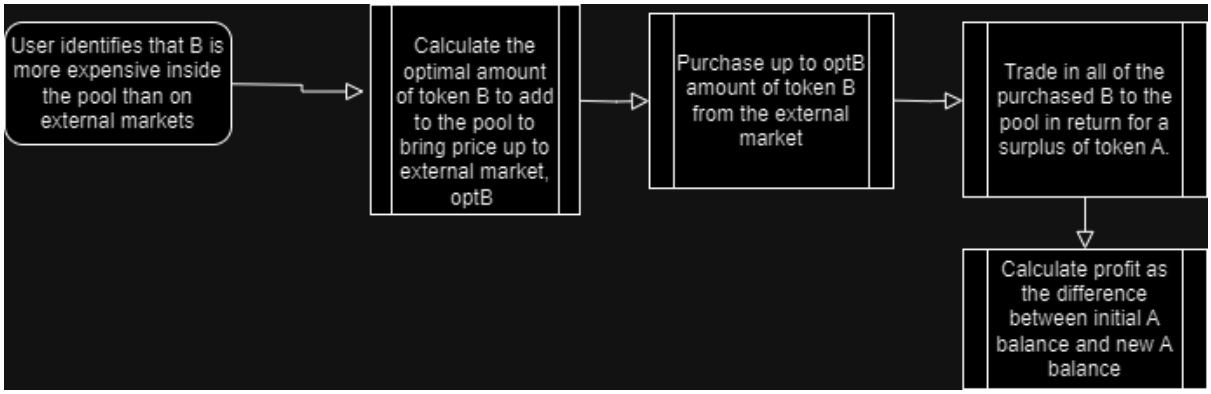
$$x * y = k \; so \; y = \frac{k}{x} \sim y_{new} = \frac{k}{x_{new}}$$

Now that we have y_{new} and we already have y_0, we now want the difference between them, this is the amount of token B that we are going to swap into the pool, i.e.:

$$y\Delta = y_1 - y_0$$

The arbitrageur should now swap up to $y\Delta$ quantity of token B into the pool for token A, then sell that to an external market for a surplus of token B.

3.2.4 Token B more expensive inside the pool: User has token A:



This arbitrage situation is the same as the 2nd situation (Token A cheaper inside the pool), but this time around we are considering the situation where the arbitrageur has some amount of token A initially (rather than token B).

Here, the arbitrage opportunity is to buy token B cheap from the external market (with token A) and sell it to the pool for a higher price than we bought it. We solve for the optimal amount of token B to add to the pool that will bring the pool price for A down to match external markets.

<u>Calculations</u>

We define two variables:

1. $y\Delta = optimal\ amount\ of\ token\ B\ to\ add\ to\ the\ pool$
2. $y_{new} = y_0 + y\Delta$

The calculations for this arbitrage situation are very similar to that of arbitrage situation 1, but this time we are solving for an optimal y value rather than an optimal x value.

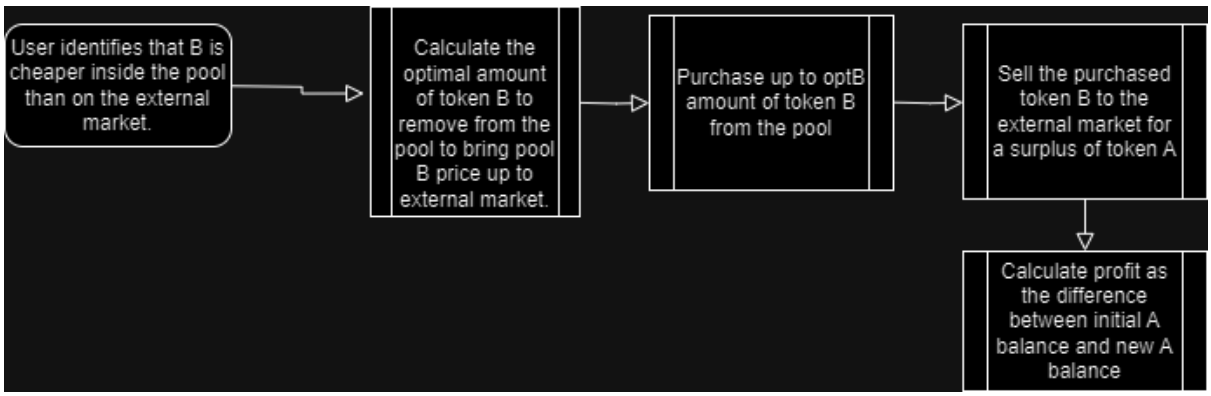
From the base formula $x * y = k$ we know that $x = \frac{k}{y}$ or in our case $x = \frac{k}{y_{new}}$.

We setup an equation that equates the external ratio P to $\frac{y_{new}}{x_{new}}$ and solve for the value of $y\Delta$.

We end up with the value $y\Delta = \sqrt{kp} - y_0$ where y_0 is the current quantity of token B prior to the swap executing.

With the optimal value of B to purchase from the external market ($y\Delta$) we now purchase up to $y\Delta$ token B from the external market (with our token A), then sell it to the pool in return for a higher amount of A than we initially had. The difference between our initial A balance and balance after the trade is our profit.

3.2.5 Token B cheaper inside the pool.



The final arbitrage situation that we consider is the situations where token B is cheaper inside the pool than on external markets. Like arbitrage opportunity 3, this opportunity happens at the same time as situation 1 (when A is more expensive inside the pool).

Here the arbitrage opportunity is to buy token B from the pool (for token A) and then sell it to the external market for profit. Here, we solve for the optimal amount of token B to remove from the pool to bring the price of B up to match the higher external ratio.

<u>Calculations</u>

We define two variables:

1. $y\Delta = optimal\ amount\ of\ token\ B\ to\ remove\ from\ the\ pool$
2. $y_{new} = y_0 - y\Delta$

We setup the same equations as we did in arbitrage situation 3 and solve for $y\Delta$.

We end up with a value of $y\Delta = y_0 - \sqrt{kp}$ where y_0 is the current quantity of token B prior to the swap executing.

Now that we have the optimal value of B to remove from the pool, we now need to figure out how much A we'll be swapping in to remove $y\Delta$ amount of B from the pool (remember, trades within our pool are subject to price impact. To do this we take a similar approach to situation 2.

We have $y\Delta$, this means we can calculate the new reserve state of token B (y_0) after the trade to be $y_{new} = y_0 - y\Delta$, by the constant product rule this allows us to solve for the new reserve state of token A after the trade (x_{new}) .

$$x * y = k \; so \; x = \frac{k}{y} \sim x_{new} = \frac{k}{y_{new}}$$

With the value of x_{new} and x_0 (which is just the old quantity of token A), we can solve for the correct amount of x to swap in ($x\Delta$) easily with $x\Delta = x_{new} - x_0$.

The arbitrageur should swap in at most $x\Delta$ amount of token A to the pool (in return for token B). Then they sell this token B to the external market in return for a surplus of token A.

3.3 Note on arbitrage opportunities:

In the above sections (3.2) I detailed the four different arbitrage opportunities that can be exploited by arbitrageurs given the design of AMMs. Its important to note that in 3.2.2 and 3.2.3 I assumed that the user had some amount of token B initially, while in 3.2.4 and 3.2.5 I assumed they were starting with some amount of token A. All four opportunities are still exploitable regardless of which token you start off with, the order of operations just differs slightly as you may have to swap between A and B with the pool or external market at different times.

3.4 Impact of arbitrage on the pool and base CPMM's vulnerability to this

In section 3.2 I detailed the four main arbitrage situations that can be exploited within the pool. Each time an arbitrageur executes an arbitrage trade, some amount of monetary value is being siphoned away from the pool (and therefore Liquidity Providers) and being taken by the arbitrageurs. The regular CPMM algorithm has no mechanism itself to 'defend' against this, and as a result LPs can experience large amounts of impermanent loss if either of the two tokens in the pool experience price change.

3.4.1 Example of arbitrage trade and impermanent loss for LP:

For this example, I am going to show an example of an arbitrage trade using the method outlined in section 3.2.2 (Token A more expensive inside the pool than on an external market). This will demonstrate how arbitrageurs are exposed to impermanent loss due to price changes in token A / token B in external markets.

<u>Setting up the pool:</u>

Assume we have a liquidity pool using the CPMM algorithm. The pool contains the token pair A and B, with an internal ratio of 1.8:1 (B:A). With this ratio, our pool has a total of 1,000 token A and 1800 token B in the reserves.

<u>Liquidity Provider adds liquidity</u>

A liquidity provider comes along (call them LP), and deposits liquidity in the pools given ratio, providing a total of 300 token A and 540 token B (in-line with the pools ratio and therefore no price change with this deposit). At the time of deposit, the external market price for token A is $9 and token B is $5, this means that the pool price and external market price are in sync. At deposit time, the value of the LPs deposit is ($9 X 300) + ($5 x 540) = $5400. The new state of the reserves after this liquidity deposit is 1300A and 2340B

Liam Michel 6641319

External market price of A falls from $90 down to $80

When the market price of A falls from $9 -> $8, there is now an arbitrage opportunity that people can take advantage of, specifically situation 1 (where A is more expensive inside the pool than on external markets).

Arbitrageur operations

Suppose an arbitrageur comes around with some amount of token B on hand, they will now calculate the 'optimum' swap to make against the pool with token A, purchase at most this amount of A from the external market, and sell it to the pool for a profit in the form of token B.

Calculation

$$x * y = k$$
$$1300 * 2340 = 3{,}042{,}000$$

The formula for 'optimal A out' value is:

$$x\Delta = \sqrt{\frac{k}{p}} - x_0$$

We need to find out the external markets value for P: Given that Token A's price has changed to $8, and Token B's price has stayed at the original $5, we now have a B:A ratio of 1.6.

We solve the above equation with values of:

- $k = 3{,}042{,}000$
- $p = 1.6$
- $x_0 = 1300$

This gives us the value of 78.85 which we will round down to 78 for safety. This means that the maximum amount of A we should swap into the pool to exploit the price difference here is 78. We buy 78A from the external market using our token B, given the price ratio of 1.6, we need $78 * 1.6 = 124.8$) token B to purchase 78 A.

The arbitrageur then swaps this 78A into the pool in return for an amount of token B. We can calculate the returned amount using the following calculation:

$$(x + dx)(y - dy) = k$$

$$(y - dy) = \frac{k}{x + dx}$$

$$y - dy = \frac{xy}{x + dx}$$

$$dy = \frac{ydx}{x + dx}$$

Given our value for dx (amount of X coming into the pool) is 78, we can solve for dy and get a value of 130 (when including the fee). Note that the fee appears 'larger' here due to the way that Solidity handles fixed point arithmetic and rounding (explained later).

The arbitrageur has now received 130 token B in return for the A they swapped in, and they purchased that A for 124 token B. This means the arb has made a total profit of (130-124 = 6) token B profit from this trade. On the external market this means a profit of $30 (from external B market price).

If the liquidity provider removed their liquidity from the pool at this point, they'd receive 317 token A and 509 token B (compared to their original deposit of 300 and 540). If we calculate the external market value of this, we get ($8 x 317) + ($5 x 509) = $5081.

If the user had just held their tokens (rather than put them in the pool), then their original investment would have been worth ($8x300) + ($5 x 540) = $5100. Given that the external market price of their tokens dropped, they were guaranteed to lose some value, but by providing liquidity to our pool and falling victim to arbitrage, they have lost even more value.

3.5 Motivations behind my implementation and solution

The above example in section 3.4 demonstrates how providing liquidity to pools with volatile token pairs has an inherent risk as each bit of price movement gives arbitrageurs an opportunity to siphon value out of the pool. Pools that use the CPMM algorithm use a 'transaction fee' that is normally somewhere between 0.3-0.5% of the amount being swapped into the pool. This transaction fee is then added to the token reserves which is the algorithms way of compensating LPs for providing liquidity.

My idea for my implementation is to add a variable transaction fee that is adjusted according to the difference between the internal token ratio and external ratio. By increasing the trade fee during times of arbitrage (i.e. when external ratio! = internal ratio), we can reduce the impermanent loss experienced by LPs by reducing the potential for arbitrage profit (without eliminating it).

The fee being variable means that it can also come back down towards the standard baseline of 0.3% during times of minimal / no arbitrage , which will help encourage high trading volume (and therefore more fee accrual).

4. System Requirements and Specification

This chapter of the report aims to cover the scope of my project. Here I will cover the system requirements for my solution and its implementation. This will include functional and non-functional requirements that were pre-defined before the project was executed.

4.1 System core aims.

The core functionality of my system implementation involves being able to deposit liquidity in return for trading fees, and to swap an amount of token A for an amount of token B. This will involve the implementation of the algorithm in Solidity smart contracts, as well as a basic front-end for the purposes of visual demonstration in the presentation.

My main contribution is to attempt to minimize impermanent loss with the integration of a Chainlink price oracle to track the external market token ratio.

There are two main types of requirements in my project, functional and non-functional. Before we can define the different requirements in our project, we need to understand the different types of requirements. These are function and non-functional requirements.

4.2 Functional Requirements

Functional requirements are the requirements that cover the core functionality of the system. A functional requirement is generally related to an integral technical feature. The functional requirements for my project are:

Smart contract Functional requirements

- Ability for LPs to add liquidity to the pool and receive the appropriate number of LP tokens in return.
- A functioning 'liquidity locking' period, before which LPs are not able to withdraw their liquidity. If they attempt to remove their liquidity before this locking period then the transaction will revert and at most, they will incur a gas fee (depending on the implementation of the check).
- Ability for LPs to remove liquidity from the pool by burning their LP tokens and receiving back their share of the reserves in return.
- Ability for users of the pool to swap an amount of token A for an amount of token B (and vice versa)
- Integrated logic in the smart contract using Chainlink to mitigate impermanent loss using a variable fee mechanism which adjusts the fee according to the difference between the pool price and external price.
- A demonstrable benefit from the fee mechanism i.e. reducing impermanent loss by a non-trivial amount.
- Proper enforcement of permissions on the contracts various methods to enforce security. Achieve this using ownership and modifiers.

Front-end Functional requirements

- Ensure that all the core functionality of the smart contract is readily and easily accessible to users.
- Integrate MetaMask to allow people to interact with the smart contract with their own wallet.

4.3 Non-Functional Requirements

Non-functional requirements are more concerned with the quality / usability of the system rather than the core functionality. We assume that the functional requirements are met, non-functional ones are more optional and not integral to the core functionality.

Smart contact Non-Functional requirements

The smart contract does not have any non-functional requirements as it is essentially the core of the system. Every requirement for it is inherently functional in nature.

Front-end Non-functional requirements

For the front-end of my system, the non-functional requirements are generally either user experience related or visual.

- Only allow calling of smart contract functions (i.e. liquidity deposit, liquidity removal and swaps) if all fields are appropriately filled.
- Implement a reactive and interactive front-end that enhances the user experience.
    - Asynchronous update of core smart contract information such as reserves, current 'pool price' etc.

4.4 Key technologies to be used in front-end.

4.3.1 Choice of language and development environment for smart contract development

As discussed previously in the introduction and literature review, this project is being implemented on the Ethereum blockchain. This means that naturally my language of choice for smart contract development is Solidity.

When writing 'traditional' applications in a language like Java or C++, it is normal to use an IDE (Integrated Development Environment) to develop and run your software locally as you develop it. Due to the nature of smart contracts and the way that you must interact with them, local development on your own machine can be rather complicated and unwieldy.

When I began this project, I started by writing the smart contracts in my browser using the Remix IDE. Remix is an online IDE for developing Solidity smart contracts. It has many of the same features as a traditional IDE such as syntax highlighting but also allows for things like contract deployment on simulated test-nets etc. Developing the various contracts for my project in Remix was not a bad experience, but frequently re-deploying the various contracts in the correct order was unwieldy and slow so I ended up searching for an alternative option.

I ended up using Hardhat for most of the development and all the testing / evaluation for my contracts. Hardhat is a JavaScript library for Ethereum development that offers all the same features that Remix did while instead working on my own machine. This meant I was able to do my development in VS Code like I normally do. One of the best features of Hardhat (and Remix) was the local simulated blockchains that they allow you to run. When testing my contracts I was often deploying and redeploying the same set of 2-3 contracts again and again. If I were to deploy these on an actual testnet each time, then each transaction would take time to be mined and it would cost me testnet ETH (which is not infinite).

I wrote a lot of unit tests for my contracts during development, all of these were written in JavaScript using Hardhat, as well as various scripts that I used for rough experimentation and the actual evaluation of my project at the end. The scripting feature with hardhat was extremely useful as it allowed me to write some basic deployment scripts that could automate a set of contract deployments, then run some pre-defined (or random) transactions all with one basic command, which was much faster and efficient than what I had to do when using Remix.

4.3.1.1 Note about Testing with Chainlink

Chainlink oracles are only accessible on actual testnet or mainnet networks. Since I developed my contracts locally with the hardhat development environment, I didn't have access to any Chainlink oracles during testing. For this reason, when testing my contracts I had to essentially mock the oracle calls by inserting my own mock values (instead of an actual Chainlink contract call). This was not an issue during actual testnet deployment though.

4.3.2 Choice of language and development environment for front-end application development

As discussed in the literature review, a large barrier to entry for De-Fi and crypto in general is the complication of initial setup and general interaction with the blockchain. For this reason, it's very useful from a user experience perspective to have an easy-to-use and interactive front-end website that abstracts away a lot of the complication for them.

While it is technically possible to interact with my liquidity pool smart contract directly without the use of the website, several important 'quality of life' features will not be there making the entire experience quite unwieldy. The main one is the automatic adjustment of token A / token B quantities when adding liquidity to the pool. This calculation would be automatically handled on the front-end for the user.

I developed the entire front-end on my local machine using the VSCode (Visual Studio code) editor. I used GitHub for version control which came in extremely useful if I ever happened to break something in my code and had to revert to a previous version. It also allowed me to keep a relatively detailed log of the development process.

4.3.3 Choice of web technologies for frontend-demo

I have experience using JavaScript from an internship in the past and have dabbled with the React JavaScript front-end library in the past. Given this fact, and the need for responsiveness in my application I chose to use React for the UI.

Picking a framework for the server-side part of my website was relatively easy given my desire to use React. NextJS is a JavaScript framework that directly integrates React components making the link between front-end and back-end extremely easy to work with. Notably, NextJS has support for 'server actions' which allows client components (i.e. rendered on the frontend) to make asynchronous calls to the server to offload computation.

React components are extremely useful for my website as it mainly consists of reactive input boxes that need updating based on a combination of user inputs and values that are read from the smart contract. Reacts state management and context hooks made managing these interactions quite easy and was very intuitive.

Another crucial thing I had to consider with my web application was the integration of MetaMask to allow people to use their own wallets. Fortunately, MetaMask has a React SDK that makes integrating it into my website extremely intuitive.

Due to above detailed reasons, I found that React combined with NextJS was the best option for my front-end implementation for this project.

5 System Design, Proposed Approach and Implementation

In this section of the report, I will discuss the planning and logic behind the system design, and the various steps I took during the development process (smart contract development).

5.1 Solidity Fixed point arithmetic

As mentioned in section 4, my implementation involves using the regular CPMM algorithm along with a variable fee mechanism that adjusts according to external market conditions to address impermanent loss caused by arbitrage. As such, I first had to develop the regular CPMM algorithm in solidity.

5.1.1 Solidity fixed point arithmetic

The main difficulty of implementing the CPMM algorithm in Solidity was the way that solidity handles floating point numbers.

Many high-level programming languages like Python and JavaScript have robust support for floating point arithmetic, for example if you perform the calculation $\frac{3}{2}$ then you will get the value 1.5. This kind of capability is extremely useful for basic scripting, but floating-point arithmetic can be very dangerous when applied to certain tasks/fields (such as finance).

5.1.1.1 Risks of floating-point arithmetic

```
a = 0.1
b = 0.2
c = 0.3
a+b == c
False
a
0.1
b
0.2
c
0.3
```

The main risk of floating-point arithmetic is the inherent imprecision of floating-point numbers in binary. For example, the floating-point number 0.3 cannot be represented 100% accurately with a computer (as shown above). We know that $0.1 + 0.2 == 0.3$ however due to the way in which computers store numbers, they are unable to accurately represent this value.

This imprecision can have catastrophic consequences, especially in fields such as finance as each bit of imprecision or rounding error can build up and end up costing thousands of dollars.

5.1.1.2 Fixed point arithmetic as a solution

Solidity addresses this issue of floating-point imprecision by instead using fixed point arithmetic. The core idea behind fixed-point arithmetic is to scale up all numbers in all calculations by some agreed scale (1e18 is the standard) and perform calculations with those

scaled values to avoid ever dealing with decimals. This approach ensures precision and consistency, which is crucial for financial applications (like liquidity pools), and other domains where accuracy is paramount.

A basic example would be the handling of adding decimal values. If we wanted to write some program that accurately added decimal values with two decimal places (i.e. 0.01 + 0.05) then we would have to apply a scale of 10^2 to bring the values up to 1 and 5.

5.1.1.3 Imprecision of 2 decimals

In regular fiat currencies such USD (US Dollar) and GBP (Great British Pound), the lowest denomination of the currencies are 1 cent and 1 penny, each of these are equal to 0.01 of their respective currencies. For regular everyday payments and transactions this is not necessarily an issue, but only being able to go down to 0.01 of a currency sets a hard limit on the 'divisibility' of the currency. For example, if we have two items, one of them is worth £0.01 and the other one is worth 50% more, we can't say it's worth £0.015, we must round up or down. This inability to go lower than 0.01 of a dollar/pound is an issue that Ethereum addresses directly with fixed point arithmetic.

5.1.2 Ethereum 18 decimals

Ethereum addresses this problem of both floating-point imprecision and low divisibility by using the 18 decimals approach. While USD is only divisible down to $\frac{1}{100}$, Ethereum is divisible down to $\frac{1}{1e18}$, that is, the smallest amount of Ethereum you can send to somebody is 10^-18 Ethereum. The smallest unit of Ethereum (10^{-18}) = 1 wei, while 1 Ether = 10^{18} wei.

Ethereum allows for a high level of divisibility compared to traditional fiat currencies, thanks to its ability to handle up to 18 decimal places. This precision significantly reduces rounding errors. ERC20 tokens, which follow the same standard (though it can be adjusted), benefit from this level of divisibility as well.

In Solidity, the programming language used for Ethereum smart contracts, floating-point arithmetic is achieved through a scaling approach. Solidity does not support floating-point numbers directly. Instead, it uses integers and scales them. For example, when sending the smallest unit of a token (1 * 10^-18), the value '1' is sent. Conversely, when sending 1 token, the value '10^18' is sent. Therefore, within the contract, a quantity of '1' represents 10^-18 of the token.

5.2 Implement CPMM In Solidity

The first thing to do for my implementation was implementing the base CPMM algorithm accurately in Solidity. The two core functions for the base CPMM algorithm are:

1. Ability to add / remove liquidity to the pool in return for a proportional amount of LP tokens (shares). These shares minted entitle the Liquidity provider to a proportion of both reserves.
2. Ability to Swap an amount of token A in for token B, or token B in for token A out.

It is of the utmost importance that all calculations for liquidity adding / removal and token swapping are as accurate as reasonably possible with floating point arithmetic used appropriately in all relevant areas.

I've already broken down the formulae and associated maths behind the CPMM algorithm earlier in this paper so will not delve deeper into that in this section of the paper.

5.2.1 Base CPMM fee mechanism and calculation

As mentioned earlier in the paper, the regular CPMM algorithm incentivizes people to provide liquidity with the use of a fixed swap fee that is charged for each swap that somebody executes. This fee is normally somewhere between 0.3-0.5% and is simply taken from the amount swapped into the pool before the swap executes. The application of this fee mechanism in the contract itself is shown below:

```
function calculateSwap(uint countIn, uint inReserve, uint outReserve) public pure returns(uint amountOut){
    //calculate amount of token in (with fee of 0.3%)
    uint countInWithFee = (countIn * 9970) / 10000;
    //dy = ydx / x + dx
    amountOut =  (outReserve * countInWithFee) / (inReserve + countInWithFee );
}
```

Here we can see the application of the 0.3% fee mechanism. Since solidity does not support decimal values, we cannot simply multiply countIn by 0.997 (99.7%), we must multiply it by 997 and then divide by 1000. Note that whenever Solidity performs divisions that would result in a decimal output (like above) it truncates the output (so 2.6 rounds down to 2).

5.3 Adding Chainlink support for external ratio tracking.

My enhancement to the CPMM algorithm was to use Chainlink for decentralized / verifiable tracking of the external market price (token ratio) and then adjust the trading fee to mitigate the impact that arbitrageurs can have on Liquidity Providers investment value. By tracking the external ratio and comparing it to the internal ratio, I can detect differences between them (i.e. opportunities for arbitrage) and then increase the trading fee accordingly to reduce the impact of arbitrage on the pool.

5.3.1 Key considerations for fee mechanism

It is important to note that that the variable fee mechanism is not intended to completely negate / remove all arbitrage opportunities from the pool. As discussed in the literature review, arbitrage is not just important but vital to the liquidity pools price movement (arbitrage helps to rebalance the pool to match external market conditions), and if it's not too extreme, can be mutually beneficial for both arbitrageurs and the liquidity providers assuming that there is high enough trading volume (and therefore enough fee generation).

5.3.1.1 Example of the impact of 'no arbitrage'

If I were to implement a fee mechanism that completely negated arbitrage (with extremely high fees), then any external price movement that deviated from the pool price would make the pool completely unattractive to everybody.

For example, if the pool price for some token (token A) was £5, but on an external market, the price was £4 (due to some price reduction), completely negating any arbitrage opportunity with an extreme fee would result in the pool becoming completely dead. Regular customers who intend to swap would avoid the pool due to its largely different price compared to external

markets, and arbitrageurs would have no incentive to rebalance the pool as there would be no profit for them. As a result of nobody using the pool, liquidity providers would not be earning any trading fees and so would likely begin to remove their liquidity en-masse.

5.3.1.2 Implementation of a reasonable trading fee

As demonstrated above, completely negating arbitrage is not a viable approach for this algorithm, however it is still possible to negate some of the impact of arbitrage with a reasonable fee adjustment, such that there is still an arbitrage opportunity, just a **reduced one.**

5.3.2 Logic behind the fee adjustment during times of arbitrage

Its important to be aware of the gas fee mentioned in the literature review. Each bit of computation that is performed, and each time state it mutated, the gas fee for a given transaction increases. As such, the more complex the smart contract operations are, the higher the gas fee. For this reason, I've implemented a compact and simple mechanism for the fee adjustment that is 'gas conscious' in that the mechanism maintains a constant small size to reduce unwanted high gas fees.

5.4 Tracking the external ratio over a period or transactions.

In my implementation I made the decision to 'track' the external ratio movement (relative to internal ratio) over a period, and to fetch the external ratio on a 'per swap' basis. The contract would fetch the external ratio (using Chainlink) on one of two conditions and adjust the swap fee accordingly:

1. Either on every other swap transaction i.e. on swaps 2,4,6,8 we fetch the external ratio.
2. On swaps that take place after some amount of time has elapsed since the last swap.

My reasoning for the above choice is as follow:

1. By checking the external ratio on every other swap, we reduce the number of times that potentially redundant fetches are performed. During times of high transaction volume, it might not be necessary to check the external ratio on every swap as it likely isn't changing that often. With that said, checking on every other transaction ensures that we are still checking on a semi regular basis, as we want to ensure that we are checking often enough, rather than not enough.
2. The second situation in which we check the external ratio is if a given period has elapsed since a swap was last executed with the pool. The time since the last swap is recorded on-chain with the help of blockchain timestamps, so every time somebody calls 'swap', the contract will know how long has passed since the last swap was executed against the pool.
    a. This check is very important as it's entirely possible that the pool may have extremely low transaction volume. On one swap the external ratio will be checked, a large period may elapse until the next trade, and so when the next swap is executed (per above) the external ratio would not be fetched. During the time that has passed between these two swaps, the external ratio may have

changed severely, and our pool would not adjust the fee accordingly. Adding this extra check solves that issue.

5.4.1 Recording Simple Moving Average (SMA) of external token ratio:

As discussed above, it is vital that the pool is reactive to the state of external markets, I've ensured this with checking external ratios every other swap and after a certain period. While this is great, its also important that the pool is not overly sensitive to small fluctuations. The way I addressed this issue was by recording the past five external ratio fetches and using the mean of those ratios for the fee calculation.

Example

If the pool has fetched the external ratio 5 times with values [2, 2, 2.1, 2.1, 2.05] then the average here is 2.05 so this would be the 'average external ratio' that we'd use for the swap fee calculation.

This mechanism helps to somewhat stabilize the transaction fee as it won't be overly reactive to tiny fluctuations in price, while also helping to mitigate the impact of large erratic changes as we are taking the average of the past five ratios. Note that once we have recorded the past five ratios, we then pop the ratios from the beginning of the list so that the oldest ratios are removed first.

5.5 Rationale behind higher trading fee during times of arbitrage

As stated in section 5.3.1.1, the aim of this mechanism is not to negate arbitrage (as its vital to the pools functionality), it is instead to mitigate the impact of it. In the Problem Analysis section, I discussed the concept of Impermanent Loss, which is a direct result of arbitrageurs rebalancing the pool to align the pool price of tokens with the external market.

When a trader swaps in some token A for some token B (or visa versa), the amount of B they receive is dependent on the amount of A they put in as well as the trading fee.

Example:

Trader gives in 1000 token A, with a 0.3% fee:

Amount of A after fee = $1000 * 0.997 = 997$.

If the fee was instead 2%, then:

Amount of A after fee = $1000 * 0.98 = 980$

The quantity after fee deduction is then exchanged for the appropriate amount of B according to the CPMM algorithm, the B is then sent to the trader. Because of this, by increasing the trading fee during times that arbitrage opportunities are detected (i.e. external ratio ! = internal ratio), then we reduce the amount of the token returned to the arbitrageur (reduces the reserve drain), while still allowing some amount of profit to maintain the arbitrage incentive.

5.6 Variable trading fee calculation

```
function calculateNewSwapFee() public{
    require(token1_reserve > 0 && token2_reserve > 0);
    uint internal_ratio = getReserveRatio();

    uint percentageDifference = 0;
    if(average_external_ratio > internal_ratio){
        percentageDifference = (average_external_ratio - internal_ratio) *1e18 / internal_ratio;
    }else{
        percentageDifference = (internal_ratio - average_external_ratio) *1e18 / average_external_ratio;

    }
    perc_diff = percentageDifference;
    //console.log('percentage difference below');
    //console.log(percentageDifference);
    if(percentageDifference < 3*1e15){
        //console.log('minimal difference, setting base fee');
        baseFee = 3*1e15;
    }else{


        uint newFee = perc_diff * 9 / 10;
        //console.log('calculated new fee');
        baseFee = newFee;
        //if the difference is more than 50%, then we cap the fee at 50%
        if(newFee > 5e17){
            baseFee = 5e17;
        }
    }
```

For the variable trading fee calculation, I took the following approach.

As stated before, I adjusted the trading fee after every other swap, or if no swaps had been executed for a certain period. The calculation of the new trade involved the following steps:

1. Fetch a new (up to date) external market ratio using the Chainlink oracle.
2. Add this new fetched datapoint to the dataPoints array (where I store the 5 most recently fetched external ratios)
3. Calculate the average external ratio using the 5 most recent datapoints.
4. Now that I have the external market token ratio, I now compare it to the current pool token ratio and calculate the percentage difference.
    a. If the difference between internal and external ratio is < 0.3% then we stick with the base CPMM fee of 0.3% as the arbitrage opportunity here is minimal.
    b. If the difference is >0.3% then we set the trading fee to $percentageDifference * 0.9$
        i. For example, if there is a 10% difference between the pool and external market price, then we use a 9% trade fee.

5.7 Example of variable trading fee use

Here I will demonstrate an example of the variable trading fee in use, including how its triggered. For the sake of demonstrating the impact, I'll re-use the same pool scenario as I showed in the Problem Analysis section 3.4.1.

The pool conditions are as follows:

X = 1300, y = 2340, k = 3,042, 000

- $x = 1300$
- $y = 2340$
- $k = 3{,}042{,}000$
- $internal\ ratio\ B{:}A = 1.8$
- $external\ ratio\ B{:}A = 1.6$

Internal ratio B:A is 1.8, on an external market the ratio has fallen to 1.6, so an arbitrageur sees an opportunity to buy A cheap from the external and sell it to the pool for profit.

We assume here that some swaps have taken place against the pool (from regular traders or arbitrageurs it doesn't matter), and therefore the pool has fetched the external ratio change and is therefore able to adjust the fee accordingly.

Given that the fetched external ratio is 1.6 and the internal ratio is 1.8, the percentage difference between the two will be calculated to be (1.8-1.6) / 1.6 = 0.125, or a 12.5% difference between the two. Therefore, the new trading fee will be 12.5 * 0.5 = 6.25%.

Using the same 'optimal' values as calculated before, we purchase 78A for 124B from the external market then swap this 78A into the pool.

<u>Calculation(s) inside the pool:</u>

When the arbitrageur calls 'swap' with the 78A, the pool first calculates the amount in <u>with the fee</u> (removes the fee before the swap. In this case the 78A that is received into the pool will become 73A with the fee removed. This 73A is then swapped for the appropriate amount of B.

<u>Calculating amount of B to return</u>

We re-use the same formula from the problem analysis section for this calculation:

$$dy = \frac{ydx}{x + dx}$$

In this case the value for dx is 73, not 78 as this was before the fee was taken.

Therefore, we calculate $dy = 124.41$. Considering the minimal token amount used in this example, we would round down to 124. However, in practical scenarios involving significantly larger numbers, the amount after the decimal point would be accounted for (remembering precision up to 18 decimals).

Comparing the output of this to the original flat 0.3% fee we can see that the pool has returned 124 token B this time instead of the original 130.

5.8 Note on suitable token pairs for my implementation:

In the literature review I briefly mentioned the suitability of certain tokens for this algorithm, and the same is true for this liquidity pool. My ratio tracking mechanism purely tracks the price ratio between the two tokens A and B on the external market but does not account for both token prices changing.

For example, if token A is worth $2 on an external market and B is worth $1, then the external ratio B: A would be 2:1. If A's price rises while B stays the same then this is not an issue as the tracking mechanism is able to monitor this and adjust the swapping fee accordingly.

If both token A and B experience a 5% price increase on the external market such that they are worth $2.10 and $1.05 respectively, then the ratio between them is still the same i.e. (1.05:2.1 == 1:2), this is not an issue.

Issues arise when both tokens change price in different ways, which is very common with volatile token pairs. For example, if we had two ERC20 tokens A and B that both had price volatility in different directions and by different amounts, then my tracking mechanism would not be great at effectively handling the fee adjustment, so these kind of token pairs are not preferable for my implementation (or the CPMM in general), as evidenced by most pool using a Stablecoin in the pair.

5.8.1 Preferable token pairs for the pool

There are two types of ideal token pair for my implementation:

1. Stablecoin – Stablecoin:
    a. This is the most ideal token pairing for the CPMM algorithm as due to both tokens having no real price fluctuation, arbitrage is very rare. Most trades made against the pool are regular traders trying to swap from A->B or B->A. This is great for liquidity providers as their exposure to impermanent loss is essentially zero, and they can earn very low risk trading fees.
2. Stablecoin – non-stablecoin(volatile):
    a. This is the kind of token pair that my implementation is most suited for. As discussed in section 5.8, both tokens being highly volatile can be 'too much' for the mechanism to handle as arbitrage opportunities will be constant. Having one token price stable while the other fluctuates some amount is most ideal for liquidity pools using the CPMM algorithm.

6 Testing and Evaluation

6.1 Testing the smart contracts.

Given that the various smart contracts in this system would be handling real user funds (if deployed on mainnet), it was extremely important that the various contracts were rigorously tested and that they performed as expected. I also found general unit testing quite useful for debugging, as its very easy to write code thinking that it works, but well written tests will immediately flag if unexpected behaviour is occurring.

To ensure this, I wrote exhaustive unit tests in JavaScript using the hardhat development environment, these contracts included:

1. SwapToken.sol: This is the smart contract for the ERC20 tokens that will be inside the pool. Two of this contract are deployed for the pool, the first one acting as Token A, and the second acting as Token B.
    a. Given that I was essentially just using the OpenZeppelin ERC20 contract with some wrapper methods, I only wrote a few tests for this. These tests ensured that minting, burning and transferring of tokens worked properly.
2. LpToken.sol: This is the contract that I used for the LP tokens in the pool (the shares that are minted to liquidity providers). This contract is essentially the same as the SwapToken

contract (its an ERC20), but it uses the OpenZeppelin 'Ownable' modifier so that another contract can take Ownership of this contract.

   a. Unit tests for this contract were mainly aimed at testing the Ownership property. Only the 'Owner' of the contract (the liquidity pool contract itself) should have permissions to mint or burn this token to people as this token entitles people to a share of the pools reserves).

3. LiquidityPool.sol: This is the contract that contains the basic CPMM algorithm without the variable fee mechanism or Chainlink. I performed rigorous testing for this contract given the importance of the algorithm performing accurately and precisely. Tests included:
   a. Adding / removing liquidity from the pool and ensuring that liquidity locking was functional.
   b. Ensuring people can swap between A and B.
   c. Ensuring that swaps are performed precisely to ensure people receive the correct amount of the token.
   d. Tested the price movement within the pool by 'draining' a tokens reserve and observing how the returned amount kept decreasing.
4. variableFee.sol: This contract implements the CPMM algorithm the same as in LiquidityPool.sol, but this time with the added fee mechanism. Unit tests for this contract were mainly focused on the fee mechanism, and included:
   a. Ensuring that a new external ratio is fetched either every other swap, or after a period has elapsed since the last swap.
   b. Ensuring that only five external ratios are stored in dataPoints within the contract state (to reduce gas). I tested that the correct (oldest) ratio was removed from the list first as well.
   c. Tested to ensure that the variable fee only kicked in after the 0.3% difference threshold was passed. Otherwise, the original base fee of 0.3% is used.

6.2 Evaluation against requirements

Before I evaluate the actual effectiveness of my trading fee mechanism, I first want to evaluate what I've done against my requirements defined in section 4 of this report.

6.2.1 Smart contract functional requirements

My variable fee liquidity pool satisfies all the requirements as follows:

1. LPs can add liquidity and are compensated with appropriate LP tokens (tested)
2. Liquidity locking feature has been implemented.
3. LPs can remove their liquidity by burning their LP tokens in return for their share of the token reserves.
4. Traders can swap an amount of token A for token B, and an amount of token B for an amount of A (tested).
5. I have integrated a variable fee mechanism that aims to mitigate the impact of impermanent loss by monitoring the difference between pool price and external market price.
6. The effectiveness of the fee mechanism will be evaluated in the next section.
7. Permission enforcement on methods is well practiced in my contracts.

6.2.2 Front-end functional requirements

1. All the core functions of the smart contract are callable on the front-end.
2. I've integrated MetaMask so people can use their existing wallets.

6.2 Evaluation

For the evaluation of my implementation, I opted for the following approach to test the effectiveness of my solution – this was done with a custom JavaScript hardhat script 'smartArbitrage.js':

1. Initialize the regular CPMM pool and the Variable fee pool and seed them with the same amounts of liquidity.
    a. Have two liquidity providers (LP1, LP2) deposit equal amounts of liquidity to each pool and record the initial value of their deposit according to the external market price at the time of deposit (this will be used later for impermanent loss calculation).
2. Establish a given scenario between the external market and the pool. These scenarios include:
    a. External token ratio is the same as internal ratio (no arbitrage opportunity)
    b. External token ratio fluctuates by small amount above/below the internal ratio due to the volatile token price fluctuating.
    c. External ratio fluctuates by a large amount above or below the internal ratio.
3. In each scenario, run **<u>an identical large set of swaps</u> (a few thousand)** against two liquidity pools. The first pool will be implementing the regular CPMM algorithm, while the second pool uses CPMM with my variable fee mechanism implemented. The idea is that by running a large amount of swaps, the pool will have time to accrue trading fees, and the effectiveness of the fee mechanism can be tested.
4. In the set of transactions, each transaction can be **One of two types**:
    a. Regular Swap: In the context of this evaluation, a regular swap consists of a trader swapping in a small random amount of (A/B) in return for (B/A).
    b. Arbitrage trade: These trades use the formulae's outlined in the Problem Analysis section to calculate 'optimal' trades during times where arbitrage is possible. If multiple arbitrage opportunities (of the 4 outlined) are possible given the pool and market state, then one of them is randomly chose. Arbitrageurs will then swap between 50-100% of the optimal amount of the
5. During the execution of each swap, the decision of whether to do a 'regular swap' or an 'arbitrage swap' is decided on a pseudo coin-toss.
6. Once all swaps are executed, burn the shares for LP1 and LP2 to redeem their liquidity from both pools, and calculate the impermanent loss experienced because of the arbitrage. Compare the impermanent loss experienced in the regular CPMM pool and in the Variable fee pool.

6.2.1 Evaluation Metric – Impermanent loss difference between two pools

The metric I used for evaluating the effectiveness / success of my solution was to measure the impermanent loss experienced by LPs from arbitrage with both pools. As stated above, both

pools will have the exact same set of swaps executed against them (with the same quantities). The impermanent loss after the trades was calculated as follows:

1. Before swaps are executed, the USD value of the liquidity (according to external market price) is recorded.
2. After swaps are executed and liquidity is removed, the new USD value of the user's tokens is calculated using the newest external market price.
3. Impermanent loss is the difference between the value of the user's tokens after liquidity withdrawal vs if they had just held their tokens (not provided liquidity).

Note that to roughly measure the performance of my algorithm I will be comparing the impermanent loss values between the regular and variable pool. As mentioned, numerous times earlier in this paper, the aim of my solution is not to negate impermanent loss but to instead mitigate it.

As such, if my pool reduces impermanent loss by a non-negligible amount then then that will be counted as successful.

6.4 Evaluation – Scenario setup (trade volume and ratio movement)

As stated in section 6.2, the way I evaluated the effectiveness of my implementation was to run a large set of transactions against my variable fee liquidity pool. My initial plan was to try and use real transaction data from some popular liquidity pool like a Uniswap V2 pool or Uniswap / Sushiswap. It turned out that scraping large amounts of useful transaction data from a site like Etherscan was much harder than I first expected, and I wanted more control over what kind of swaps were being executed. It was also much easier to run large amounts (i.e. >1000) swaps against the pool if I created my own ones rather than scraping real-world ones.

6.4.1 Modelling trade volume with Poisson Process

For each scenario I have a count of swaps (transactions) that are to be executed, I had to find an appropriate way of modelling the transaction count at different times (different price ratios). I chose to use Poisson process for this as its well-suited to the random and independent nature of transaction arrivals, which means it can be used to generate realistic transaction counts for the different price ratios used during the scenario.

6.4.2 External price ratio modelling

For modelling the external price ratio, I used a more heuristic approach to simulate each specific scenario. For example, in the first scenario (no arbitrage), the external ratio is a fixed value.

In the second scenario (small arbitrage opportunity) I used 'random walk' which essentially takes an initial value and then takes random 'steps' above or below that value. You can adjust the size of these steps using volatility, so for this one I made steps within +- 1% of the starting ratio so that the arbitrage opportunities were relatively small.

For the third scenario (large difference between external and internal ratio), I used random walk again but with a larger level of volatility so that the price deviated further from the starting point.

6.4 Evaluation – Constant external ratio

The first scenario I ran was the situation where the external token ratio is constant (doesn't change) and the only thing changing is the pool ratio with each trade. This is essentially the 'baseline' situation that the algorithm will experience in real world use and so I thought it would be a good starting point for the evaluation.

As explained earlier in the paper, arbitrage is not only essential but an inevitable event that takes place when using the CPMM algorithm in a liquidity pool. Each swap from A->B or B->A causes small changes in the pools token ratio (token price) and so even with a 'constant' external market ratio, these small changes in the internal ratio can result in arbitrage opportunities.

My prediction for this test is that the total value of the user's liquidity stake will be slightly higher with the variable fee pool, but not by a significant amount. Given that the external token ratio is fixed, the only opportunities for arbitrage will be caused by large random swaps that might cause large amounts of price movement in the pool.

6.4.1 Constant external ratio results

```
Account: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  Pool 0:
    Old Total: 3876427.4707920129622016
    New Total: 3902533.28693020818767734
    Difference: 26105.81613819522547574
    Gain
  Pool 1:
    Old Total: 3876427.4707920129622016
    New Total: 3911232.84825668241152201
    Difference: 34805.37746466944932041
    Gain

Account: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
  Pool 0:
    Old Total: 3856945.11323651573284864
    New Total: 3882919.72536067623320275
    Difference: 25974.61212416050035411
    Gain
  Pool 1:
    Old Total: 3856945.11323651573284864
    New Total: 3891575.56396420119530418
    Difference: 34630.45072768546245554
    Gain

Account: 0x90F79bf6EB2c4f870365E785982E1f101E93b906
  Pool 0:
    Old Total: 1996399.44604868155539456
    New Total: 2009844.20601636728000628
    Difference: 13444.75996768572461172
    Gain
  Pool 1:
    Old Total: 1996399.44604868155539456
    New Total: 2014324.56829423834483388
    Difference: 17925.12224555678943932
    Gain

Account: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65
  Pool 0:
    Old Total: 823657.86263635525369856
    New Total: 829204.7897708826311443
    Difference: 5546.92713452737744574
    Gain
  Pool 1:
    Old Total: 823657.86263635525369856
    New Total: 831053.26033869997308537
    Difference: 7395.39770234471938681
    Gain
```

```
Account: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  Pool 0:        Regular CPMM pool
    Old Total: 3876427.4707920129622016
    New Total: 3902533.28693020818767734
    Difference: 26105.81613819522547574     Profit / loss for this LP
    Gain
  Pool 1:        Variable fee pool
    Old Total: 3876427.4707920129622016
    New Total: 3911232.84825668241152201
    Difference: 34805.37746466944932041
    Gain
```

The above image shows the results from the first scenario. For each scenario I am using four liquidity providers, but this is purely to demonstrate the gain/loss depending on how much liquidity a user deposited into the pool.

In the results, pool 0 is the 'regular CPMM liquidity pool', while pool 1 is the CPMM liquidity pool with the variable trading fee added.

We can see that both pools perform relatively well, with LPs receiving profits from investing in both pools. For example, the first account receives a profit of £26105 from the accrued trading fees in pool 0, note that while this may sound like a lot of money, it's very small compared to the almost £4 million of liquidity they provided to the pool.

The most important thing to consider is the difference in profit between the two pools. For account 1 (first account), pool 0 generates £26105 in profit while pool 1 with its variable fee (charging higher fees at times of arbitrage) generated nearly £35000 in fees. That is almost a 50% increase in trading fee generation for this user which is a big percentage difference in the context of trading fee profits.

6.5 External ratio changes by small amount from baseline value

The second scenario I tested was where the pool starts off with an internal ratio B:A of 2.0, while the market ratio is 1.97. Every few transactions the external price would then shift by a small amount using 'random walk' as mentioned in section 6.4.2.

6.5.1 Small amount from baseline value results

```
Account: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  Pool 0:
    Old Total: 2880779.88306064359373342902
    New Total: 2899651.76530070128494438617
    Difference: 18871.88224005769121095715
    Gain
  Pool 1:
    Old Total: 2880779.88306064359373342902
    New Total: 2908400.4306254409806405475
    Difference: 27620.54756479738690711848
    Gain

Account: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
  Pool 0:
    Old Total: 1307151.49086518266992615406
    New Total: 1315714.59183328404881203956
    Difference: 8563.100968101378885855
    Gain
  Pool 1:
    Old Total: 1307151.49086518266992615406
    New Total: 1319684.29149328173835502737
    Difference: 12532.80062809906842887331
    Gain

Account: 0x90F79bf6EB2c4f870365E785982E1f101E93b906
  Pool 0:
    Old Total: 1762995.02389247651330073941
    New Total: 1774544.33894995260736319554
    Difference: 11549.31505747609406245613
    Gain
  Pool 1:
    Old Total: 1762995.02389247651330073941
    New Total: 1779898.39377514455747036306
    Difference: 16903.36988266804416962365
    Gain

Account: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65
  Pool 0:
    Old Total: 2063356.51592736552237088113
    New Total: 2076873.48798649637485662465
    Difference: 13516.97205913085248574352
    Gain
  Pool 1:
    Old Total: 2063356.51592736552237088113
    New Total: 2083139.71322279966723325638
    Difference: 19783.19729543414486237525
    Gain
```

In this scenario, we have both the external and internal ratio changing. The external ratio change is done manually by me with random walk, while the internal ratio changes due to price movement from swaps.

The results of this scenario are very similar, with both pools generating decent trading fees for all LPs, with the variable fee pool generating more fees than the base CPMM pool. Since the results are very similar, I will try one more ratio change before possibly changing my approach to evaluation.

6.6 External ratio changes by large amount from baseline value

The third scenario I tested was the situation where the pool starts off with an internal B:A ratio of 2.0, whereas the external market has a ratio of 1.6. This represents a 20% difference between the ratios which is an extremely large price difference for arbitrageurs to take advantage of.

6.6.1 Large amount from baseline value results

```
Account: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  Pool 0:
    Old Total: 1395069.42083455273402368
    New Total: 1405906.84499066552734748
    Difference: 10837.4241561127933238
    Gain
  Pool 1:
    Old Total: 1395069.42083455273402368
    New Total: 1439268.89219257242445304
    Difference: 44199.47135801969042936
    Gain

Account: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
  Pool 0:
    Old Total: 2004963.55582880404996096
    New Total: 2020538.8670983080882442875
    Difference: 15575.3112695040382833275
    Gain
  Pool 1:
    Old Total: 2004963.55582880404996096
    New Total: 2068486.07874864243143836
    Difference: 63522.5229198383814774
    Gain

Account: 0x90F79bf6EB2c4f870365E785982E1f101E93b906
  Pool 0:
    Old Total: 2305318.76097378188525568
    New Total: 2323227.3444855411933474475
    Difference: 17908.5835117593080917675
    Gain
  Pool 1:
    Old Total: 2305318.76097378188525568
    New Total: 2378357.32738849949697993
    Difference: 73038.56641471761172425
    Gain

Account: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65
  Pool 0:
    Old Total: 2605611.27474219605557248
    New Total: 2625852.64348600984667102
    Difference: 20241.36874381379109854
    Gain
  Pool 1:
    Old Total: 2605611.27474219605557248
    New Total: 2688163.8984239669248691225
    Difference: 82552.6236817708692966425
    Gain
```

The results of this scenario are more interesting than the last one as we can observe a large difference in the trading fee accrual between the two pools. In section 6.4 and 6.5 we could observe a fee accrual difference of ~50% between the regular and variable fee pool. This time

around we can see that the variable fee pool is accruing ~400% of the fees that the regular pool is. While this result is initially very good, it does raise some questions about the sustainability of this as such a large difference in fee generation is somewhat suspicious and potentially unrealistic in actual practice.

6.7 Hyper-Aggressive arbitrage test

The final test I wanted to run against my variable fee mechanism was a 'hyper-aggressive' arbitrage situation. In the earlier scenarios, arbitrage trades were executed on a coinflip basis with sometimes less than optimal trades being made, as well as regular non-arbitrage trades being sprinkled into the mix. Especially given the results of section 6.6, its important to evaluate the performance of the fee mechanism when its under the most realistic load.

For this scenario I made three changes:

1. Arbitrage trades are now executed whenever they can be assuming there is more than a 0.01% ratio difference.
2. Arbitrage trades are now executed with optimal values at all times.
3. Regular (non-arbitrage) trades are now executed with smaller amounts of A/B

6.7.1 Hyper-Aggressive arbitrage results

```
Account: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  Pool 0:
    Old Total: 1000378.82386948173070330735
    New Total: 1002006.92980213997210916116
    Difference: 1628.10593265824140585381
    Gain
  Pool 1:
    Old Total: 1000378.82386948173070330735
    New Total: 1002804.78696799506803909444
    Difference: 2425.96309851333733578709
    Gain

Account: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
  Pool 0:
    Old Total: 3759492.64221850316861875324
    New Total: 3765611.17664619666493644953
    Difference: 6118.53442769349631769629
    Gain
  Pool 1:
    Old Total: 3759492.64221850316861875324
    New Total: 3768609.5789244169078047493
    Difference: 9116.93670591373918599606
    Gain

Account: 0x90F79bf6EB2c4f870365E785982E1f101E93b906
  Pool 0:
    Old Total: 3040419.26935732940585171553
    New Total: 3045367.51949227522607513495
    Difference: 4948.25013494582022341942
    Gain
  Pool 1:
    Old Total: 3040419.26935732940585171553
    New Total: 3047792.42118289433341027476
    Difference: 7373.15182556492755855923
    Gain

Account: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65
  Pool 0:
    Old Total: 1708513.54497444043467425674
    New Total: 1711294.13266005378710845627
    Difference: 2780.58768561335243419953
    Gain
  Pool 1:
    Old Total: 1708513.54497444043467425674
    New Total: 1712656.76623674792818165617
    Difference: 4143.22126230749350739943
    Gain
```

The idea behind the changes listed above were to try and simulate a more aggressive arbitrage situation where more value was being siphoned from the pool and less 'regular swaps' were being executed. The results for this test are likely more representative of the mechanisms real world performance.

We can see that pool 1 (variable fee) gathers around 50% more in trading fees than the regular CPMM pool (1600 vs 2400, 6000 vs 9000) etc. This result is very different to the scenario ran in section 6.6 where the variable fees were generating 400% that of the regular pool. By introducing more 'optimal' arbitrage trades, increasing the overall volume of them while simultaneously reducing the number of 'regular swaps', I've increased the aggression of the arbitrage severely.

Despite the 'lower' performance compared to 6.6, the fee accrual is similar to that in section 6.4 and 6.5 (which performed well) which implies that the model works well in this more realistic setting.

6.7 Evaluation findings and Conclusion

From the above scenarios that I've executed with my arbitrage script I've concluded that my implementation is effective but within certain limits, and impermanent loss will still be noticeable for token pairs that are overly volatile (though I've addressed this in an earlier section).

The fee mechanism performed well under all 4 scenarios:

1. Fixed external ratio: The fee mechanism kicked in whenever regular swaps caused the internal price ratio to deviate from the fixed external ratio, garnering slightly higher fees than the regular CPMM pool.
2. External ratio changes by small amount with random walk: This had very similar results to the first scenario as the two tests are functionally the same. In scenario 1 (fixed external ratio), the internal ratio still moves as swaps cause price movement and so the fee mechanism is activated. This scenario is very similar and so the results were very similar.
3. External ratio largely different to internal ratio: This scenario saw the variable fee pool massively outperform the regular pool in fee accrual. This was likely due to the higher fee being charged due to the higher percentage difference between the ratios.
4. Aggressive arbitrage: This scenario involved optimal arbitrage trades at all times, and arbitrage trades whenever one was possible. The results for this were positive as they were very similar to scenario 1 and 2. Scenario 3 showed a surprisingly high amount of fee accrual to the point of it being unsustainable (i.e. arbitrageurs would not be happy with that high a fee being charged). Its also somewhat unlikely that there would be such a large ratio difference between the pool and external market as in scenario 3.

For the above reasons I conclude that my mechanism is an overall success within the earlier discussed boundaries (i.e. 'stablecoin – volatile token' or 'stablecoin – stablecoin' pairs.

7 Statement of Ethics

7.1 Justification of idea

The ultimate 'goal' of this project is to reduce the loss of monetary value from people's investments in liquidity pools. To that end, my project is inherently a force for good as I'm only

trying to benefit those who have lawfully invested their funds into a liquidity pool and are trying to innocently earn interest on their funds while providing liquidity for swaps.

Despite the inherent risks of smart contract code (i.e. undetected vulnerabilities, malicious code, exit schemes), none of my smart contracts do anything malicious and so by no means is my code inherently immoral / illegal or unethical.

On the other hand, given that my project directly deals with people money (completely takes hold of user funds), there are possibly some moral, legal and ethical implications to be discussed and how exactly I've ensured they are addressed in my implementation.

7.2 Legal Considerations

Despite the financial nature of my project, given that I did all development and testing either on my own local blockchain using Hardhat and only ever deployed contracts on the Sepolia testnet (fake money), no real user funds were ever touched during this project.

An important aspect of this project is that it does not actively harm society as a whole or specific members of the public, and that it respects privacy, legality and confidentiality.

Regarding the legality of my project, its very important to consider CMA (Computer Misuse Act 1990) and the BCS code of conduct. CMA was bought about with the aim of 'protecting the integrity and security of computer systems and data through criminalising access to them which has not been authorized by the owner of the system or data'[9]. My project as its been implemented does not directly interface with any client machines without their expressed permission / consent, and does not unknowingly read or manipulate any data.

7.3 Ethical considerations – Informed Consent

Informed consent is a very important thing to include when considering the ethics of this project, especially given that its in the field of finance. In the context of my project specifically (liquidity pools), it is important that all users interacting with the pool (traders, liquidity providers) are fully aware of the risks and potential losses/gains that they will experience when using the pool. Specifically, this includes understanding core concepts such as slippage, impermanent loss, and possible vulnerabilities in the smart contract code that could lead to complete loss of funds.

Given that none of the smart contracts in my project are ever being deployed on mainnet, this is not a large issue as nobody stands to gain or lose any money by interacting with my project. On the other hand, even though my project is only operating in controlled test environment, there is still an ethical obligation to provide accurate and up-to-date information regarding the internal mechanics and state of the pool (this includes things such as token reserve values, current trading fee etc). If this information was not publicly available then this would be an issue both operationally (as arbitrageurs would be unable to perform relevant calculations), but also ethically as LP's and traders alike would be unaware of the state of the pool they're investing with.

7.4 Positive contribution to Society

The final ethical consideration I will make for this project is how it is ethically and legally composed. For my project to be ethically and legally composed, it must have a net-positive effect on society (which I believe it does give that its aim is to save investors money). My project

does not present any harm or actively harm anybody, and it handles user data properly (in the few small ways that it does handle user data).

7.5 Ethics (closing)

Throughout the duration of this project, I actively tried to carry it out in an ethical, legal and socially acceptable manner. Although there were few legal matters/principles to consider given the theoretical and testnet / local development environment everything took place in, I have tried to carry out the project in such a way that if my project were ever deployed on the Ethereum mainnet, that all relevant laws would be followed.

Regarding my front-end implementation, any interaction that the users have with it are safe and do not involve the unlawful / unauthorized storage of any private user information that could cause the users any harm (for example my frontend code never accesses their wallets private keys as I am using MetaMask).

References

1. https://www.investopedia.com/what-went-wrong-with-ftx-6828447 (FTX Collapse scandal)
2. https://www.investopedia.com/articles/forex/121815/bitcoins-price-history.asp#:~:text=The%20pandemic%20shutdown%20and%20subsequent,the%20start%20of%20that%20year.
3. https://arxiv.org/ftp/arxiv/papers/2204/2204.08664.pdf
4. SoK: Decentralized Finance (DeFi) Sam M. Werner∗ , Daniel Perez∗ , Lewis Gudgeon∗ , Ariah Klages-Mundt† , Dominik Harz∗‡, William J. Knottenbelt∗ ∗ Imperial College London, † Cornell University, ‡ Interlay
5. https://www.investopedia.com/tech/cryptocurrency-burning-can-it-manage-inflation/
6. https://chain.link/education-hub/oracle-problem
7. https://www.legislation.gov.uk/ukpga/1990/18/section/1 CMA (1990)
8. https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ BCS Code of Conduct
9. https://www.gov.uk/government/consultations/review-of-the-computer-misuse-act-1990/review-of-the-computer-misuse-act-1990-consultation-and-response-to-call-for-information-accessible#:~:text=The%20Act%20has%20the%20intention,of%20the%20system%20or%20data. Review of the CMA 1990
10. https://www.openzeppelin.com/contracts OpenZeppelin open-source contracts for ERC20 token